

xLSTM Architecture Feasibility For Recommendations

Author: Vivekanand

(This work was conducted as part of academic coursework)

Abstract

The primary idea of Recommender system is to present users with the most relevant items such as Music, Movies, Software, Services by learning large scale interaction datasets. In this thesis, we will investigate the feasibility of Extended Long Short Term Memory (xLSTM) architecture for sequential recommendation tasks.

We benchmark xLSTM against strong existing baselines, including Transformer based models (such as BERT4Rec, SAS4Rec) and recurrent alternatives, on five public datasets: MovieLens 100K, MovieLens 1M, MovieLens 10M, MUSIK4all, and Amazon Software. We also evaluate and compare models using top k ranking metrics (Recall@10, MRR@10, NDCG@10) with training time and further conduct an in depth analysis on the model architectural embedding weights and performance results.

Our results show that, xLSTM attains competitive accuracy relative to Transformer baselines, particularly on medium and large scale sequential datasets and while offering favorable memory scaling and stable training. On smaller sequences, performance gaps narrow or favor Transformers models, highlighting the trade offs between capacity and data sequence settings.

Overall, our results indicate that xLSTM is a strong sequence model at scale competitive with Transformer baselines on larger sequential like MovieLens ML-1M/10M, memory efficient and training stable, yet bit less effective on the smallest sequences like MovieLens ML 100K. Finally, these results and findings will point to a practical path for deploying xLSTM in modern large scale sequential recommender systems.

Kurzfassung

Ziel von Recommender Systemen ist es, Nutzerinnen und Nutzern auf Basis großer Interaktionsdatensätze möglichst relevante Inhalte wie Musik, Filme, Software oder Dienstleistungen bereitzustellen. In dieser Masterarbeit wird die Eignung der Extended Long Short Term Memory Architektur (xLSTM) für sequentielle Empfehlungsaufgaben untersucht.

Hierzu wird xLSTM mit etablierten und leistungsstarken Baseline Modellen verglichen, darunter transformerbasierte Ansätze wie BERT4Rec und SAS4Rec sowie rekurrente Alternativen. Die Evaluation erfolgt auf fünf öffentlichen Datensätzen: MovieLens 100K, MovieLens 1M, MovieLens 10M, MUSIK4all und Amazon Software. Neben klassischen Top k Ranking Metriken (Recall@10, MRR@10, NDCG@10) werden auch Trainingszeiten analysiert. Zusätzlich wird eine detaillierte Untersuchung der architektonischen Einbettungsgewichte sowie der erzielten Leistungsunterschiede durchgeführt.

Die experimentellen Ergebnisse zeigen, dass xLSTM im Vergleich zu Transformer Baselines eine konkurrenzfähige Genauigkeit erreicht, insbesondere auf mittelgroßen und großen sequentiellen Datensätzen, bei gleichzeitig günstiger Speicherkomplexität und stabiler Trainingsdynamik. Bei kleineren Sequenzen verringern sich die Leistungsunterschiede oder fallen zugunsten der Transformer Modelle aus, was auf einen Zielkonflikt zwischen Modellkapazität und Datencharakteristik hinweist.

Insgesamt belegen die Ergebnisse, dass xLSTM ein leistungsfähiges und skalierbares Sequenzmodell darstellt, das auf größeren Datensätzen wie MovieLens 1M/ MovieLens 10M mit transformerbasierten Ansätzen konkurrieren kann und zugleich speichereffizient sowie trainingsstabil ist. Auf sehr kleinen Datensätzen wie MovieLens 100K zeigt sich jedoch eine leicht geringere Effektivität. Die gewonnenen Erkenntnisse liefern somit eine fundierte Grundlage für den praktischen Einsatz von xLSTM in modernen, großskaligen sequentiellen Recommender Systemen.

Contents

Abstract	iii
Kurzfassung	iv
List of Tables	vii
List of Figures	viii
List of Acronyms	ix
1 Introduction	1
1.1 Motivation	1
1.2 Classification of Recommender Systems	2
1.3 Problem Statement	3
1.4 Research Questions and Contributions	4
2 Background Related Work	5
2.1 Existing Model Architectures For Sequential Recommenders	5
2.1.1 Recurrent Neural Networks (RNN)	5
2.1.2 Long Short Term Memory (LSTM)	6
2.1.3 Self-Attentive Sequential Recommendation (SAS4Rec)	8
2.1.4 Bidirectional Encoder Representations for Recommendation (BERT4Rec)	10
2.2 Existing Literature Review and Architectures	11
3 Proposed Method: xLSTM for Sequential Recommendation	14
3.1 xLSTM Introduction	14
3.2 Architecture, Training objective inference	15
3.3 Model Architectures Compared	18
4 Experimental Setup	20
4.1 Methodology	20
4.1.1 Datasets	20
4.1.2 Model Pipeline	21
4.1.3 Model Selection	22
4.1.4 Data Preparation and Splitting Protocol	22
4.1.5 Training Configuration	23
4.2 Evaluation Metrics	24
4.2.1 Loss Functions and Hyperparameters	26
4.2.2 Cold Start and Sparsity Handling	28
5 Results	29
5.1 Experimental Results and Interpretation	29
5.2 Comparative Performance Results and Analysis	29
5.3 Results in Response to Research Questions	29

5.4	Scaling Characteristics and Computational trade offs	32
6	Discussion	34
6.1	Key Observations on Model Parameters	34
6.2	Predictions and Qualitative Insights	35
6.3	Embedding Geometry Analysis for Sequential Recommenders . . .	36
6.4	Model Inference Speed Comparison	42
6.5	Fairness, Popularity Bias and Diversity Analysis	43
6.6	Runtime Performance, Resource Usage and Logging	47
6.7	Transfer Learning in Latent Factor Models	48
6.8	Industrial Applications	48
6.9	Limitations and Considerations	49
7	Conclusion	50
	Bibliography	51

List of Tables

- 3.1 Model architecture comparison across key dimensions. 19
- 4.1 Experimental datasets. 21
- 4.2 xLSTM and DataLoader hyperparameters per dataset. 27
- 5.1 Performance comparison across models, datasets and sequence lengths (excluding training time, LR and hit rate). 30
- 6.1 Layer wise Trainable Parameter Breakdown for xLSTM Model Across MovieLens Datasets with Item Counts and Memory Estimates 35
- 6.2 Popularity Bucketed Metrics Across Models and Datasets 46

List of Figures

- 1.1 Recommender Classifications 3
- 2.1 Bert4Rec Model Architecture 10
- 3.1 xLSTM Architecture 15
- 3.2 xLSTM Architecture Proposed 17
- 4.1 Model Pipeline 21
- 5.1 Comparision Results 31
- 6.1 Sequence Predictions 36
- 6.2 L2 Normalization 39
- 6.3 Mean and Variance per embedding dimension 39
- 6.4 Spectrum Hubness Analysis 41
- 6.5 Cosine Structure and CKA Similarity 42
- 6.6 Inference Time Scaling Analysis 44
- 6.7 Popularity Bias 45
- 6.8 Popularity Chart 45
- 6.9 GPU Training Performance Graph 47

List of Acronyms

xLSTM Extended Long Short-Term Memory

SR Sequential Recommendation

BERT Bidirectional Encoder Representations from Transformers

BERT4Rec BERT for Recommendation

BiLSTM Bidirectional Long Short-Term Memory

SVD Singular Value Decomposition

MRR@10 Mean Reciprocal Rank at 10

NDCG@10 Normalized Discounted Cumulative Gain at 10

Recall@10 Recall at 10 items

GPT Generative Pre-trained Transformer

GRU Gated Recurrent Unit

GRU4Rec GRU for Recommendation

LLMs Large Language Models

LSTM Long Short Term Memory

RNN Recurrent Neural Network

SAS4Rec Self Attentive Sequential Recommendation

Chapter 1

Introduction

1.1 Motivation

The rapid growth of the user interaction data has created an unprecedented demand [39] to create efficient sequential recommender systems that must act faster, accurate and computationally efficient. Despite there are several advancements in the sequential recommendations research which are primarily based on existing architectures like LSTM [11] and Transformer [4] based, but still the scalability, long term dependency and computational problems remain the key challenges [36]. This motivates us to the exploration of the latest enhanced architectures like xLSTM [1] and to address the existing limitations also to check the feasibility of sequential recommendation research for the large scale interaction datasets.

The critical subdomain of recommenders is sequential recommendations, which models user item interactions over the period of time [36], capturing temporal patterns and evolving user preferences. Recent advancements in this research area have leveraged deep learning architectures, including Recurrent Neural Networks (RNN)[34], Long Short Term Memory (LSTM)[11] and other recent architectures like BERT4Rec [4] and SAS4Rec [5], which primarily adopt Transformer based self attention mechanisms for sequence modeling.

In this thesis, we will investigate xLSTM, primarily mLSTM [1], an enhanced variant of LSTM that incorporates the architectural improvements such as bidirectional processing, a fully parallelizable matrix valued memory cell with a covariance based update rule and the refined gating mechanisms[1]. We will also benchmark xLSTM against the classical and state of the art existing sequential recommenders using datasets like MovieLens and other temporal interaction datasets. Our final results shows that xLSTM demonstrates notable improvements due to its hybrid design including mLSTM and sLSTM that blends recurrent memory and attention mechanisms, which we will analyze further.

To provide the overview of existing architecture, Recurrent Neural Networks (RNN), Long Short Term Memory (LSTM) were primarily used to predict the user preferences and to provide us with the personalized recommendations. [2, 10]. The classical methods such as collaborative filtering and content based filtering will have also evolved to incorporate sequential modeling, capturing dynamic user behavior over the period of time [14].

Among the deep learning approaches, Long Short Term Memory (LSTM) network remains a widely used architecture for modeling sequential interactions

due to its ability that it can preserve long term dependencies and manage temporal dynamics effectively utilizing its gating mechanisms.

Despite its success in implementation, LSTM also suffered from several well known limitations, such as: (1) its inability to revise storage decisions once made - meaning that once the LSTM decides whether to keep or forget some information, it cannot go back and change that decision later (2) the need to compress all contextual information into scalar cell states - that all the memory must be compressed into single vector, a fixed size memory and (3) limited parallelizability due to its recurrent memory mixing [1] - it can process one step at a time and hard to perform computations in parallel. These limitations have led to development of enhanced variants, such as xLSTM, which will introduces architectural refinements like attention mechanisms, bidirectional memory and improved gating functions to better handle large scale sequential recommendation tasks.

1.2 Classification of Recommender Systems

Recommender systems can be broadly classified into four major categories [22] based on the nature of data used and task formulation:

1. **General Recommendation (GR):** These models rely solely on user item interaction data, typically implicit feedback [33]. Implicit feedback will contains the datasets which indirectly indicate user preferences such as clicks, Add to cart events, purchase, Time spent, frequency etc. They are evaluated through top-N recommendation tasks and primarily align with collaborative filtering approaches.
2. **Content Aware Recommendation:** These models incorporate additional side information such as user or item features. They are often applied in click through rate (CTR) prediction tasks, using explicit feedback and binary classification evaluation. Its feature based, often go beyond raw user item interactions about users, items or context.
3. **Sequential Recommendation (SR):** SR focuses on next item prediction by modeling the temporal ordering of user interactions. These models utilize sequential data to capture evolving user preferences. RNN based and transformer based models are generally included in this category and this is the **primary research focus in this thesis work**.
4. **Hybrid and Other Knowledge based Recommendation:** These methods leverage external knowledge graphs to enhance general or sequential recommendations by providing semantic or structural context beyond user item interactions.

Recommendation systems can also be categorized into two paradigms: Collaborative Filtering, which models similarities between users or items based on interaction histories and content based Filtering, which relies on item attributes or metadata to make predictions [14, 10].

Collaborative Filtering can further be classified into two types described in Figure 1.1 Recommender Classifications flow diagram. Memory based methods compute similarity between users or items using metrics like cosine similarity

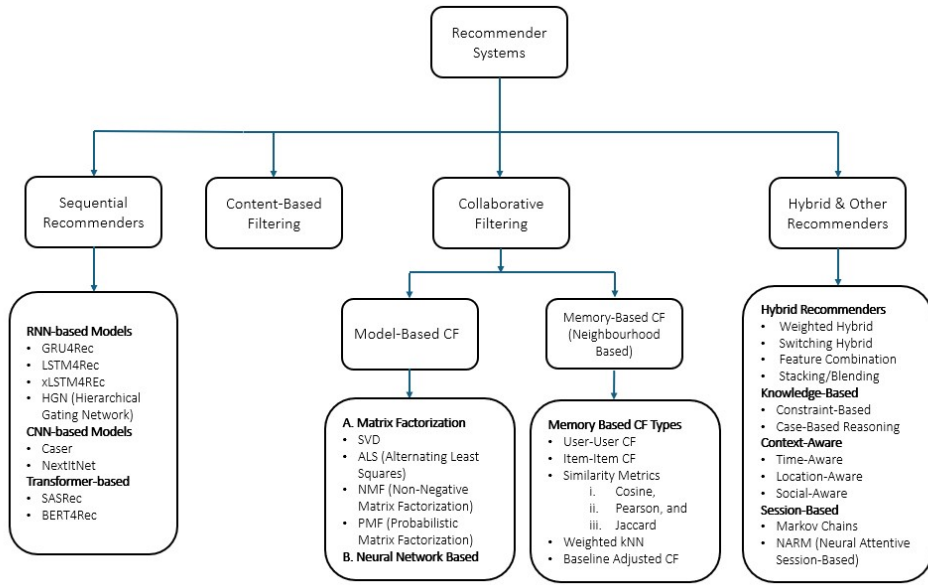


Figure 1.1: Recommender Classifications

or Pearson correlation to generate recommendations. Memory based relies directly on the rating matrix, while model based approximates it through learned parameters. Model based methods learn latent factors or prediction functions using the algorithms such as matrix factorization, SVD [32], or neural networks. Model based approaches typically achieve better accuracy and scalability [40, 43, 41].

1.3 Problem Statement

Despite the advancements brought by deep learning in improving recommendation accuracy, several critical challenges persist in modeling user item sequences at scale. These challenges include the limited scalability, popularity biases, cold start problem, data sparsity, transfer learning and the need for robust long term dependency tracking across user interactions [17, 2].

Recurrent architectures such as LSTM, which is effective in capturing temporal dependencies, but suffer from limited scalability, limited parallelizability and are inherently sequential which makes it less efficient for large scale recommendation environments. [19, 18]. In contrast, Transformer based models like BERT4Rec and SAS4Rec offer greater parallelism and improved performance through self attention mechanisms but often come with high computational costs and memory requirements problems. [4, 5].

In this thesis, we investigate whether the recently proposed xLSTM architecture [1], which integrates bidirectional memory, attention mechanisms and enhanced gating structures, can effectively balance the scalability, computational efficiency and high accuracy in sequential recommendation tasks and help us to resolve the existing challenges which we have discussed above.

1.4 Research Questions and Contributions

This thesis aims to answer the following four key research questions:

- **RQ1:** How does xLSTM’s performance scale with dataset size compared to established architectures like BERT4Rec and Sas4Rec ?
- **RQ2:** How do sequence length and embedding size influence model performance across different item popularity levels and do larger sequences or embeddings improve a model’s ability to make accurate long tail (less popular) recommendations?
- **RQ3:** What trade offs exist between recommendation accuracy and computational cost as sequence length and model complexity increase?
- **RQ4:** Embedding Saturation and Utilization: How different model architectures make effective use of their embedding representations and does model embedding dimensionality lead to better spatial distribution of embeddings, representation diversity, or predictive performance?

The primary objective is to answer these four RQ’s and evaluate the effectiveness of the xLSTM model across multiple datasets and benchmark it against state of the art baselines using established ranking metrics.

The goal of this work is to perform next item prediction by modeling sequential user item interactions within a personalized recommendation framework. We also make following key contributions in the field of sequential recommender systems:

Implementation and Evaluation of xLSTM for sequential recommendation: We adapt the xLSTM model for next item recommendation and implement it using user item interaction data. We describe the main architectural choices needed to apply xLSTM in practical recommender system settings[1]. We also present a novel adaptation and configuring the latest available xLSTM model, detailing the architectural choices needed for practical deployment on user item interaction data.

Comprehensive Model Benchmarking: We evaluate xLSTM across five public datasets and compare it with leading sequential models, including SAS4Rec and BERT4Rec[4]. The comparisons use standardized top-k ranking metrics and controlled experimental settings, addressing performance scaling (RQ1) and sequence length effects (RQ2).

Analysis of computational efficiency and scaling behavior: We examine model stability, training time, inference characteristics and resource consumption to understand accuracy–efficiency trade offs (RQ3), particularly under long sequences and large item movie predictions.

Embedding utilization and representation analysis: We conduct an in depth study of embedding saturation, dimensionality behavior and latent space organization across models, offering new insights into how different architectures use embedding capacity (RQ4). Also, discuss interaction sparsity, long range dependency modeling and popularity imbalance.

Chapter 2

Background Related Work

2.1 Existing Model Architectures For Sequential Recommenders

2.1.1 Recurrent Neural Networks (RNN)

Recurrent Neural Networks read a user's history in order (one step after another) to predict the next item[34]. They capture short term trends (what the user is into right now), but training can be unstable and slow on long sequences. It shows the need for more stable, deeper sequence models, laying foundations for xLSTM and RQ3 in the proposed architecture.

Vanilla RNN [34] updates its hidden state by combining the current input with the previous state, then it will predicts the next item, it is simple and fast but struggles with long range dependencies due to vanishing gradients [34, 37, 42, 38].

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h), \quad (2.1)$$

$$s_{t+1} = E^T h_t, \quad (2.2)$$

$$\hat{y}_{t+1} = \text{softmax}(s_{t+1}), \quad (2.3)$$

$$\mathcal{L} = - \sum_{t=1}^{T-1} \log \hat{y}_{t+1}[y_{t+1}], \quad (2.4)$$

Where $x_t \in \mathbb{R}^d$ is the input item embedding at time t (vector), $y_t \in \{1, \dots, N\}$ is the ground truth next item index at time t (scalar), $h_t \in \mathbb{R}^d$ is the hidden state at time t (vector), $\hat{y}_{t+1} \in \mathbb{R}^N$ is the predicted probability distribution over all items (vector), $\phi(\cdot)$ is a pointwise nonlinearity (e.g., tanh or ReLU) applied element wise (function), $E \in \mathbb{R}^{N \times d}$ is the shared item embedding/output matrix (matrix), $W_x \in \mathbb{R}^{d \times d}$ and $W_h \in \mathbb{R}^{d \times d}$ are learnable weight matrices (matrices), and $b_h \in \mathbb{R}^d$ is a bias vector (vector).

The hidden state h_t is updated with each new input x_t and the next item prediction is made by comparing the hidden state h_t with the item embeddings matrix

E , producing the output probability distribution $\hat{y}_{t+1}$. The model aims to predict the next item by maximizing the likelihood of the true next item, y_t , through the Cross entropy loss.

Next item prediction: The model uses the input item embedding x_t and updates the hidden state h_t . After processing the entire sequence, it compares the hidden state with the item embeddings matrix E to predict the next item. This is done by computing a probability distribution $\hat{y}_{t+1}$ over all possible items.

Cross entropy loss: The model optimizes this distribution using the Cross entropy loss, which measures how close the predicted distribution $\hat{y}_{t+1}$ is to the true next item y_t .

Simple RNN recurrent structure with a single hidden state updated at each time step. It suffers from vanishing gradients, limiting long term dependency learning for sequential recommendation tasks. [37]. RNN limitations directly justify exploring improved recurrent architectures such as xLSTM and addressing RQ3 by illustrating the accuracy–efficiency trade offs of early sequential models.

2.1.2 Long Short Term Memory (LSTM)

LSTM [11] enhanced sequential learning further by remembering what standard RNNs forgot, making it a cornerstone for early sequence aware recommenders in the previous section. LSTM serves as the architectural predecessor to xLSTM, directly informing RQ1 (scaling) and RQ2 (sequence length). LSTM is an advanced RNN variant with better gates controlling information flow to capture long range dependencies and mitigate vanishing gradient issues [11] (previously discussed limitations with RNN). Here, LSTM improves on RNNs with gates that help the model to remember important past interactions for longer. It's a strong sequential baseline [11] model used widely before the Transformers, offering better long range memory.

However, it can be heavier to train and may still lag behind attention models on very long histories. LSTMs extend RNNs with a gated memory cell that preserves information over long spans, mitigating vanishing gradients and enabling Next item predictors to remember earlier parts of a user's sequence. [11]

LSTM has three gates in its design: 1. Forget gate which decides what information to discard from the cell state, 2. Input gate which determines what new information to add to the cell state and Output Gates which controls what information is output from the cell state as the hidden state[11]. Sigmoid activation in the gates (forget, input, output) to control the flow between 0 and 1 and tanh activation for updating the cell state and output.

The range of all three gates is between 0 and 1, due to sigmoid activation function, which outputs values in the range (0,1). The values determine how much information passes through each gate, where 0 means “block all information,” and 1 means “allow all information”[11].

element wise multiplication (also called the Hadamard product) in its gating mechanisms. This means that each element of the gate vector (values between 0 and 1 from the sigmoid activation) multiplies the corresponding element of another vector (such as the previous cell state or the candidate cell update). this operation allows gates to selectively scale each component of the cell state

or hidden state, controlling how much information to keep or discard for each dimension independently[11].

LSTMs add a gated memory cell to overcome vanishing gradients and keep long range context in user sequences [11, 35].

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad (2.5)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (2.6)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad (2.7)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c). \quad (2.8)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (2.9)$$

$$h_t = o_t \odot \tanh(c_t). \quad (2.10)$$

$$s_{t+1} = E^\top h_t, \quad (2.11)$$

$$\hat{y}_{t+1} = \text{softmax}(s_{t+1}). \quad (2.12)$$

The weight matrices $W_i, W_f, W_o, W_c \in \mathbb{R}^{d \times d_x}$ correspond to the input to hidden connections between the current input vector $x_t \in \mathbb{R}^{d_x}$ and the input gate, forget gate, output gate and candidate cell state, respectively. The matrices $U_i, U_f, U_o, U_c \in \mathbb{R}^{d \times d}$ denote the recurrent (hidden to hidden) connections between the previous hidden state $h_{t-1} \in \mathbb{R}^d$ and the respective gates and cell input. The bias terms $b_i, b_f, b_o, b_c \in \mathbb{R}^d$ are learned vectors. The functions $\sigma(\cdot)$ and $\tanh(\cdot)$ denote the logistic sigmoid and hyperbolic tangent activation functions applied element wise, where the sigmoid gates control information flow and tanh squashes values to a bounded range. The cell state $c_t \in \mathbb{R}^d$ is a vector that acts as the network's memory, enabling preservation of long term information, while the hidden state $h_t \in \mathbb{R}^d$ is a vector representing the exposed output used for prediction. Through gated additive updates, LSTM mitigates the vanishing gradient problem of vanilla RNNs and allows more stable learning over long interaction sequences. $E \in \mathbb{R}^{N \times d}$ denotes the shared item embedding (output) matrix, where each row corresponds to an item representation used to score all candidate items for Next item prediction. $y_t \in \{1, \dots, N\}$ is the ground truth next item index (scalar) and $\odot$ denotes element wise multiplication. The output head compares h_t with E to produce Next item probabilities for ranking[11].

Although LSTM does not impose a specific loss function, we can employ a softmax Cross entropy objective appropriate for Next item prediction, which is consistent with prior sequential recommendation work. For Next item recommendation, we can use a softmax Cross entropy objective that encourages the model to assign high probability to the true next item in the sequence while suppressing all other candidates[11].

$$s_{t+1} = E^T h_t, \quad (2.13)$$

$$p(y_{t+1} | h_t) = \text{softmax}(s_{t+1}), \quad (2.14)$$

$$\mathcal{L} = - \sum_{t=1}^T \log p(y_{t+1}^* | h_t), \quad (2.15)$$

Where $h_t \in \mathbb{R}^d$ is the sequence representation at time t (vector), $E \in \mathbb{R}^{N \times d}$ is the item embedding/output matrix (matrix), $s_{t+1} \in \mathbb{R}^N$ is the unnormalized score vector for all items (vector), $p(y_{t+1} | h_t) \in \mathbb{R}^N$ is the predicted probability distribution (vector), $y_{t+1}^* \in \{1, \dots, N\}$ is the ground truth next item index (scalar), T is the sequence length and $\mathcal{L}$ is the training loss (scalar).

For a sequential datasets like MovieLens, with an interaction sequence $\langle \text{ItemID}_1, \text{ItemID}_2, \dots, \text{ItemID}_t \rangle$, the model processes the embeddings of the observed items to produce a sequence representation h_t . Using this representation, it computes a score vector s_{t+1} for all items in the catalog. The softmax function converts these scores into a probability distribution [11] over item IDs and the training loss penalizes the model if the true next movie ItemID_{t+1} is assigned low probability. By minimizing the loss $\mathcal{L}$ across all users and time steps, the model learns to rank the correct next item higher than other candidates.

LSTM's memory bottlenecks [11] are closely tied to our RQ1 (scaling) and RQ2 (sequence length effects). The main limitations of LSTMs in sequential recommendations are: difficulty capturing very long term dependencies accurately, high computational cost with a slower training on large sequential datasets and challenges in the interpretability and hyperparameter sensitivity, which we will address in the upcoming methods.

2.1.3 Self-Attentive Sequential Recommendation (SAS4Rec)

SAS4Rec [5] originally uses the self attention mechanisms to effectively capture long term user patterns or context in sequential recommendation tasks, which is important for evaluating competitiveness in RQ1 and RQ3. Unlike the traditional Markov Chains and RNN based approaches, SAS4Rec adaptively focuses on the most relevant items in a user's history, allowing it to balance long range dependency modeling with computational efficiency. The architecture employs Transformer style self attention blocks, positional embeddings to preserve the order of items and incorporates residual connections [5] and layer normalization to stabilize training. SAS4Rec outperform the sequential models on both the sparse and dense datasets, demonstrating the better accuracy and being significantly faster due to its parallelizable design similar to BERT4Rec (which is discussed in the next section). The model dynamically adjusts its attention span depending on dataset density, focusing narrowly on recent actions in sparse data and broadly on several past actions in denser data[5]. The visualizations of attention weights provides insights into how the model captures meaningful sequential patterns that drive recommendations. This approach marks a significant advancement in efficient, interpretable and accurate sequential recommendation systems.

SAS4Rec models user sequences using masked self attention. Given the sequence embedding matrix $E_b \in \mathbb{R}^{L \times d}$, the self attention operation is defined as

$$SA(E_b) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + \mathbf{M}\right)V, \quad (2.16)$$

$$Q = E_b W^Q, \quad K = E_b W^K, \quad V = E_b W^V, \quad (2.17)$$

Where $\mathbf{M}$ is a causal mask that prevents attending to future positions, $E_b \in \mathbb{R}^{L \times d}$ denotes the sequence embedding matrix for a user, with sequence length L and embedding dimension d . The matrices $W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$ are learned projection matrices that map input embeddings to the query, key and value spaces, respectively. The resulting matrices $Q, K, V \in \mathbb{R}^{L \times d}$ represent the query, key and value vectors for all positions in the sequence. The term $QK^\top \in \mathbb{R}^{L \times L}$ computes pairwise similarity scores between all item positions, which are scaled by $\sqrt{d}$ for numerical stability. The matrix $\mathbf{M} \in \mathbb{R}^{L \times L}$ is a causal (lower triangular) mask that assigns $-\infty$ or a very large negative number to future positions, ensuring that each item attends only to past interactions. The $\text{softmax}(\cdot)$ function is applied row wise to produce attention weights and the final output $SA(E_b) \in \mathbb{R}^{L \times d}$ is a matrix of context aware item representations used for Next item prediction[5].

The relevance score between item i and the hidden representation at time t is computed as

$$r_{i,t} = \mathbf{h}_t^{(b)} \mathbf{M}_i^\top, \quad (2.18)$$

where $\mathbf{h}_t^{(b)}$ is the output of the final self attention block and $\mathbf{M}_i$ denotes the item embedding.

SAS4Rec is trained using a binary Cross entropy loss with negative sampling:

$$\mathcal{L} = - \sum_{t=1}^n \left[\log \sigma(r_{o_t,t}) + \sum_{j \in \mathcal{N}_t} \log \sigma(-r_{j,t}) \right], \quad (2.19)$$

Where t indexes the position in the user interaction sequence, $o_t \in \{1, \dots, N\}$ is the ground truth next item at position t (scalar item ID), $\mathcal{N}_t$ is a set of sampled negative item indices (scalars), and $\sigma(\cdot)$ denotes the logistic sigmoid function. The score $r_{i,t} \in \mathbb{R}$ is a scalar relevance score for item i at time t , computed as $r_{i,t} = \mathbf{h}_t^\top \mathbf{e}_i$, where $\mathbf{h}_t \in \mathbb{R}^d$ is the context aware hidden representation produced by the masked self attention layer, and $\mathbf{e}_i \in \mathbb{R}^d$ is the embedding vector of item i . This objective encourages high scores for the true next item while suppressing scores for sampled negative items[5].

Transformer style self attention to the recommendation domain, which models user item sequences without recurrence, allowing for full parallelism and effective long range dependency capture[5]. The model uses positional encodings and attention weights to identify the most relevant past items when predicting the next interaction, offering high accuracy and scalability, analyzed in RQ1 and RQ3.

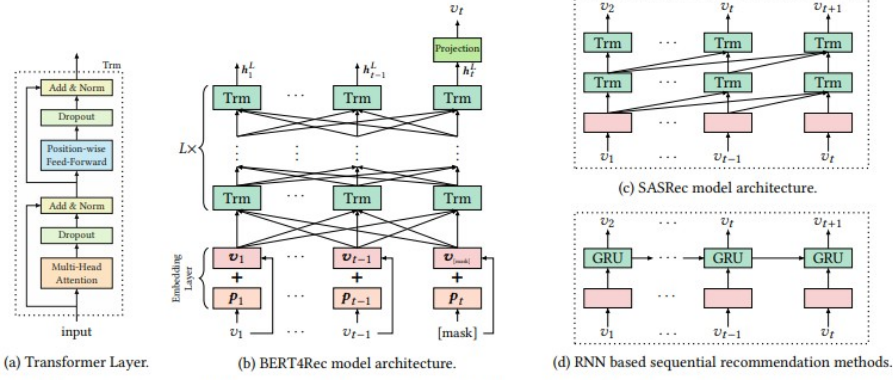


Figure 1: Differences in sequential recommendation model architectures. BERT4Rec learns a bidirectional model via Cloze task, while SASRec and RNN based methods are all left-to-right unidirectional model which predict next item sequentially.

Figure 2.1: Bert4Rec Model Architecture

2.1.4 Bidirectional Encoder Representations for Recommendation (BERT4Rec)

BERT4Rec adopts the BERT (Bidirectional Encoder Representations from Transformers) [4] architecture (figure 2.1) for the sequential recommendation. This acts as the strongest Transformer baseline in this thesis, critical for benchmarking xLSTM (RQ1, RQ2) and comparing embedding behavior (RQ3, RQ4). It treats user interaction data as a sequence and uses a masked item prediction task to learn the bidirectional dependencies. Unlike SAS4Rec, which processes in a left to right fashion, here BERT4Rec learns from both past and future items, improving its ability to model complex sequence semantics.

RNN and SAS4Rec which encode users/items interactions from left to right into the hidden representations for the predictions[34][5]. Such unidirectional model architectures restrict the power of historical sequence representations and in order to address this challenge which BERT4Rec will address this [4].

Conditioning both left and right contexts in a deep bidirectional model can trivialize training because each item may indirectly access the target item in the sequence. To address this, a bidirectional model is trained using the Cloze task[4], which predicts randomly masked items in the sequence based on both their left and right contexts. Unlike predicting the next item sequentially, the Cloze task generates more training samples, enabling the model to learn a stronger representation and achieve consistently better performance on benchmark datasets. [4]

BERT4Rec models user behavior using bidirectional self attention and the masked item prediction objective. Its core components are defined below:

1. Scaled dot-product self attention (context mixing across the sequence).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (2.20)$$

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad (2.21)$$

Where $H \in \mathbb{R}^{L \times d}$ is the matrix of hidden representations for a length- L interaction sequence, with each row $\mathbf{h}_t \in \mathbb{R}^d$ corresponding to the context-

tual embedding of the item at position t . The matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are learned projection matrices that map hidden states to query, key and value spaces, respectively. The resulting matrices $Q, K, V \in \mathbb{R}^{L \times d_k}$ contain query, key and value vectors for all positions in the sequence. The term $QK^\top \in \mathbb{R}^{L \times L}$ computes pairwise similarity scores between all items in the sequence, scaled by $\sqrt{d_k}$ for numerical stability. The softmax function is applied row wise to produce attention weights that sum to one, enabling each position to attend to all other positions in the sequence. The output of the attention layer is a matrix in $\mathbb{R}^{L \times d_k}$ representing context aware item embeddings used in subsequent Transformer layers[4].

2. Masked-position prediction with tied item embeddings.

$$P(v | S', t) = \text{softmax}(\mathbf{h}_t^{(L)} \mathbf{E}^\top + \mathbf{b}), \quad (2.22)$$

Where $\mathbf{h}_t^{(L)} \in \mathbb{R}^d$ is the hidden representation produced by the final Transformer layer at the masked position t , encoding contextual information from the entire sequence S' . The matrix $\mathbf{E} \in \mathbb{R}^{N \times d}$ denotes the shared item embedding matrix, where each row corresponds to the embedding of one item in the catalog of size N , this matrix is tied with the input embedding layer. The bias vector $\mathbf{b} \in \mathbb{R}^N$ is a learned parameter that accounts for item specific popularity effects. The product $\mathbf{h}_t^{(L)} \mathbf{E}^\top \in \mathbb{R}^N$ produces a score for every item and the softmax function converts these scores into a probability distribution $P(v | S', t)$ over all items, representing the likelihood that item v is the correct masked item at position t [4].

3. Cloze (masked item) training objective.

$$\mathcal{L} = -\frac{1}{|M|} \sum_{t \in M} \log P(v_t^\star | S', t), \quad (2.23)$$

Where $M \subset \{1, \dots, T\}$ denotes the set of masked positions in the input sequence S' and $|M|$ is the number of masked items. The symbol $v_t^\star \in \{1, \dots, N\}$ represents the ground truth item index (scalar) at masked position t , drawn from a catalog of N items. The conditional probability $P(v_t^\star | S', t)$ is obtained from the softmax output of the Transformer encoder at position t . This objective maximizes the likelihood of recovering the true masked items given their surrounding context, enabling bidirectional sequence modeling without relying on left to right recurrence[4].

We will use Bert4Rec as a strong baseline model to compare its performance [4] against SAS4Rec and xLSTM in upcoming sections. BERT4Rec is the strongest Transformer baseline in our study, critical for evaluating xLSTM's competitiveness (RQ1). Its embedding behavior provides a meaningful comparison point for RQ4, especially regarding saturation and representational geometry.

2.2 Existing Literature Review and Architectures

To understand where xLSTM fits within the broader evolution of sequential recommenders, we review prior work that blends sequence modeling, at-

tention, LSTMs and Transformers. Each of these approaches exposes limitations or open questions that directly motivate the research direction of this thesis. This Existing literature review is structured around the four research questions (discussed in section 1.4) thereby motivating the design choices and evaluation focus of our proposed xLSTM based approach. Each paragraph synthesizes prior work relevant to one research question and highlights the remaining gaps.

Performance scaling across dataset sizes (RQ1), early sequential recommender systems predominantly relied on recurrent neural architectures such as LSTM to model temporal dependencies in user–item interactions. Works such as Noorian, Harounabadi, and Hazratifard [17] and Yang and Esquivel [19] DynaPR framework demonstrated that LSTM based models can effectively capture evolving preferences on moderate-sized datasets, but their performance degrades as interaction volume grows due to limited parallelism and sequential computation. Educational and domain-specific systems based on BiLSTM further highlight these scalability constraints when applied to large or heterogeneous datasets [18]. These findings motivate RQ1, which investigates whether xLSTM can better maintain performance as dataset size increases compared to classical LSTM and Transformer based baselines.

Sequence length sensitivity and Long Tail behavior (RQ2), Several studies emphasize modeling recent interactions using attention-enhanced recurrent architectures, particularly in session based recommendation. Kumar and Kumar [20] show that attention-driven LSTMs improve short term intent modeling but struggle with long user histories and sparse item interactions. Transformer based models such as SAS4Rec and BERT4Rec address this by modeling the global sequence context through self attention [5, 4]. However, their effectiveness varies significantly with sequence length and data density, often favoring users with long, dense histories and popular items. These observations directly inform us the focus for RQ2, which examines how sequence length and embedding capacity influence performance across different Item popularity regimes, particularly for Long Tail recommendations.

Accuracy–efficiency trade offs perspective (RQ3), While Transformer based architectures deliver strong accuracy, their quadratic computational complexity (it means how the time and memory cost grow with sequence length, for a sequence of length L and embedding dimension d , self attention in Transformer based models has time complexity $\mathcal{O}(L^2d)$ and memory complexity $\mathcal{O}(L^2)$ due to the $L \times L$ attention matrix, whereas recurrent models scale linearly as $\mathcal{O}(Ld)$.) with respect to sequence length imposes significant memory and runtime costs. In other words, Transformer has to compare every item in a user’s history with every other item at each step. As the sequence gets longer, the amount of computation and memory grows very fast—if the history length doubles, the cost becomes about four times larger in general. Generative retrieval models such as TIGER [16] further increase computational overhead by combining Transformer encoders with large output spaces, limiting their practicality in large scale or low latency environments. Conversely, recurrent models offer lower per-step complexity but suffer from slower training and limited scalability. Surveys of large language model–based recommenders also highlight the inefficiency

of applying general-purpose attention-heavy models to sequential recommendation tasks [14]. These contrasting properties motivate RQ3, which analyzes the trade offs between recommendation accuracy and computational cost as sequence length and model complexity increase.

Beyond accuracy, representation quality and embedding utilization (RQ4), beyond predictive performance, recent literature highlights structural issues in learned embedding spaces, including anisotropy, popularity-driven compression and limited effective dimensionality. Transformer based recommenders have been shown to concentrate variance into a small number of dominant directions, leading to hubness and reduced Long Tail diversity [21]. Although recurrent architectures distribute information more gradually, their limited expressiveness restricts representation diversity in complex sequences. These findings will further help us on research question RQ4, which investigates how different architectures utilize embedding capacity, whether the higher dimensional embeddings translate into the meaningful latent spaces[1] and how xLSTM further compares to Transformer baselines in terms of representation diversity and stability.

These prior work reveals that the persistent limitations across performance scalability (RQ1), long sequence robustness (RQ2), computational accuracy trade offs (RQ3) and embedding utilization (RQ4). These gaps suggest us that the need for an architecture that combines recurrent memory with attention mechanisms while remaining computationally efficient and representationally stable. The xLSTM architecture is designed to address these challenges [1] and is therefore further evaluated in this thesis with respect to the research questions RQ1 to RQ4.

Chapter 3

Proposed Method: xLSTM for Sequential Recommendation

3.1 xLSTM Introduction

Proposed xLSTM Architecture: The proposed xLSTM recommender architecture, a recurrent model that extends the LSTM by modifying both the recurrent memory representation and the gating mechanism [1]. In the memory representation, the structure of the cell state be a vector or matrix valued, whereas previously it was vector valued [11]. In the gating mechanism, the memory update functions are reformulated using the exponential gating with the normalization and stabilization techniques [1]. xLSTM integrates below two variants:

sLSTM (scalar LSTM): replaces sigmoid gates with exponential gating and normalization, enabling controlled memory reinforcement and stable long term sequence modeling [1][11].

mLSTM (matrix LSTM): enhanced storage capacities, retrieval using matrix multiplication, memory cell update from scalar to matrix, covariance update rule for storing key pair value [1][11].

xLSTM blocks are residual architectures [1] that nonlinearly summarize past sequences in a higher dimensional space to better separate and model different contexts [1]. They use either post up projection with sLSTM or pre up projection with mLSTM, depending on how memory capacity scales in high dimensions. Such architectural advancement will be utilized for evaluating the performance scaling (with respect to RQ1), Sequence length sensitivity (RQ2), computational study (RQ3) and to analyse the stored embedding weights and utilization for representation diversity (RQ4).

The architectural design of xLSTM enhances the previous standard LSTM through several mechanisms that make it more suitable for modern recommender systems:

- **Exponential Gating:** In mLSTM, the input and forget gates use exponential activation functions ($\text{range}(0, \infty)$) instead of sigmoid functions ($\text{range}(0, 1)$). Unlike sigmoid gates, which can attenuate the memory state, exponential gates will allow gate values greater than one, enabling the model to actively reinforce or revise memory contents [1] [34]. This increased flexibility supports controlled memory accumulation over long sequences. To prevent the numerical instability resulting

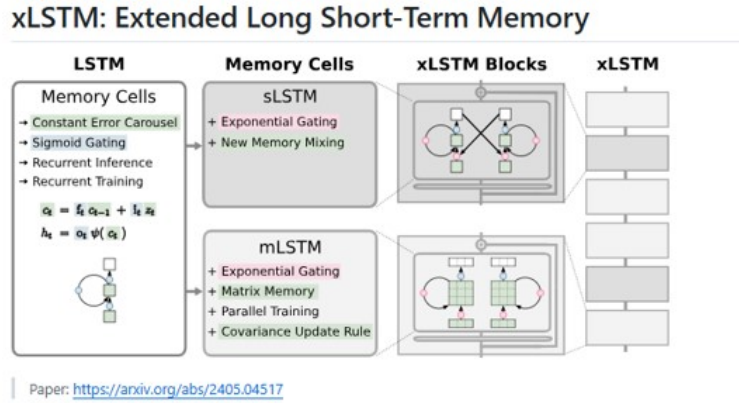


Figure 3.1: xLSTM Architecture

from unbounded gate values, mLSTM introduces explicit normalization and stabilization states that will regulate the memory growth and will also ensure the stable training dynamics (Figure 3.1) [1] [34].

- **Matrix Memory:** Instead of a vector cell state, mLSTM maintains a matrix valued memory. This enables the storage of structured information, meaning key value pairs stored via the outer products, which capture second order interactions between features rather than independent per dimension activations (Figure 3.1) [1] [34] [13].
- **Parallel Training:** The covariance update rule allows memory updates to be reformulated so that recurrence does not depend on memory mixing across time steps. As a result, the mLSTM recurrence can be expressed in a parallel form, enabling efficient parallel computation across sequence positions [1] [12].
- **Covariance Update Rule:** Memory updates are based on outer products between projected value and key vectors, accumulating second order statistics in the memory matrix that encode interactions between the sequence elements and support associative retrieval [1].

3.2 Architecture, Training objective inference

This section describes the architecture, training objective and inference procedure for our proposed model. As discussed in the previous architecture design introduction section, xLSTM make two primary modifications in its architecture, exponential gating and memory structures [1]. xLSTM contains two members: i. sLSTM[1] (with scaler memory, scaler update and memory mixing) and mLSTM [1] (matrix memory and the covariance (outer product) update rule [1]. Like the standard LSTMs, it works with memory cells and gates to control the flow of information. However, xLSTM introduces new mechanisms to improve how information is stored and transferred. The key idea is the introduction of the “cross gates,” which connect hidden and memory states more dynamically and these cross gates allow deeper interaction between different parts of the model.

- a) **Exponential input gate (new gating):** grows multiplicatively to let the cell store strong events.

$$i_t = \exp(\tilde{i}_t), \quad \tilde{i}_t = w_i^\top x_t + r_i h_{t-1} + b_i \quad (3.1)$$

- b) **Forget/output gates (optionally exponential):** control decay and exposure of memory.

$$f_t = \sigma(\tilde{f}_t) \text{ or } \exp(\tilde{f}_t), \quad \tilde{f}_t = w_f^\top x_t + r_f h_{t-1} + b_f, \quad (3.2)$$

$$o_t = \sigma(\tilde{o}_t), \quad \tilde{o}_t = w_o^\top x_t + r_o h_{t-1} + b_o \quad (3.3)$$

- c) **sLSTM cell update (scalar memory):** exponential gates write/decay a scalar state.

$$c_t = f_t c_{t-1} + i_t z_t, \quad (3.4)$$

$$z_t = \phi(\tilde{z}_t), \quad \tilde{z}_t = w_z^\top x_t + r_z h_{t-1} + b_z \quad (3.5)$$

- d) **sLSTM normalization & output:** a running “strength” normalizer keeps the ratio stable.

$$n_t = f_t n_{t-1} + i_t, \quad (3.6)$$

$$h_t = o_t \frac{c_t}{n_t} \quad (3.7)$$

- e) **mLSTM covariance (matrix memory) update:** stores key–value pairs via an outer product.

$$C_t = f_t C_{t-1} + i_t v_t k_t^\top, \quad (3.8)$$

$$q_t = W_q x_t + b_q, \quad (3.9)$$

$$k_t = \frac{1}{\sqrt{d}} W_k x_t + b_k, \quad (3.10)$$

$$v_t = W_v x_t + b_v \quad (3.11)$$

- f) **mLSTM retrieval with stabilizer:** read by multiplying the memory with a query and dividing by a gate–strength normalizer.

$$n_t = f_t n_{t-1} + i_t k_t, \quad (3.12)$$

$$h_t = o_t \odot \frac{C_t q_t}{\max(|n_t^\top q_t|, 1)} \quad (3.13)$$

Where $x_t \in \mathbb{R}^{d_x}$ denotes the input vector at time step t (e.g., the embedding of the current item), and $h_{t-1} \in \mathbb{R}^d$ is the previous hidden state vector. The scalars $i_t, f_t, o_t \in \mathbb{R}_+$ represent the input, forget, and output gates, respectively, with $\tilde{i}_t, \tilde{f}_t, \tilde{o}_t$ their pre-activation values. The parameters $w_* \in \mathbb{R}^{d_x}$ and $r_* \in \mathbb{R}^d$ are input to hidden and hidden to hidden weight vectors and $b_* \in \mathbb{R}$ are scalar bias terms. The scalar cell state $c_t \in \mathbb{R}$ stores accumulated memory, while $n_t \in \mathbb{R}_+$ is a scalar normalizer that stabilizes exponential growth. The function $\sigma(\cdot)$ denotes the logistic sigmoid, $\exp(\cdot)$ the exponential gate and $\phi(\cdot)$ a bounded nonlinearity[1].

For the matrix memory variant, $C_t \in \mathbb{R}^{d \times d}$ is the covariance (key–value) memory matrix, with $q_t, k_t, v_t \in \mathbb{R}^d$ denoting query, key and value vectors

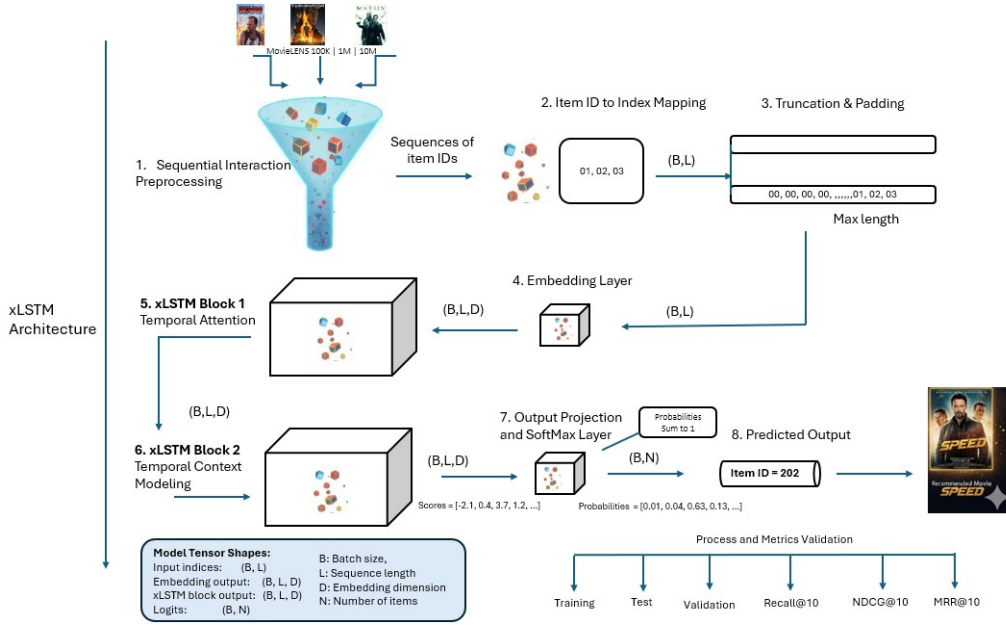


Figure 3.2: xLSTM Architecture Proposed

computed from the input via learned matrices $W_q, W_k, W_v \in \mathbb{R}^{d \times d_x}$ and biases b_q, b_k, b_v . The operator $\odot$ denotes element wise multiplication. The resulting hidden state $h_t \in \mathbb{R}^d$ is used as the representation for downstream prediction, such as Next item ranking[1].

xLSTM employs the same Next item prediction loss as LSTM and Transformer based models, the architecture differs only in how sequential representations are computed[1]. Below is the Softmax Cross entropy Loss function.

$$s_{t+1} = E^T h_t, \quad (3.14)$$

$$p(y_{t+1} | h_t) = \text{softmax}(s_{t+1}), \quad (3.15)$$

$$\mathcal{L} = - \sum_{t=1}^T \log p(y_{t+1}^* | h_t), \quad (3.16)$$

Where $h_t \in \mathbb{R}^d$ is the hidden state vector produced by the sequence model at time step t and $E \in \mathbb{R}^{N \times d}$ denotes the shared item embedding (output) matrix, where N is the total number of items in the catalog and each row corresponds to an item representation. The score vector $s_{t+1} \in \mathbb{R}^N$ contains unnormalized scores for all items at the next time step[1]. The conditional distribution $p(y_{t+1} | h_t) \in \mathbb{R}^N$ is obtained by applying the softmax function, producing probabilities over item IDs. The scalar $y_{t+1}^* \in \{1, \dots, N\}$ denotes the ground truth next item index and $\mathcal{L}$ is the softmax Cross entropy loss summed over all time steps T . By minimizing this loss over all users and sequences, the model learns to rank the correct next item higher than other candidates in Next item recommendation tasks such as MovieLens.

These architectural improvements directly target the accuracy–efficiency trade offs in RQ3. The structured memory and gating behavior serve as the basis for embedding saturation and geometric analysis in RQ4.

3.3 Model Architectures Compared

SAS4Rec[5], BERT4Rec[4] and xLSTM [1] differ fundamentally in how they transform an input sequence of item embeddings $x_t \in \mathbb{R}^d$ into hidden representations for Next item prediction. SASRec computes context aware representations using causal self attention, where queries Q , keys K and values V are linear projections of the embedding sequence E_b and a causal mask M ensures that each position attends only to past items. BERT4Rec similarly relies on self attention but operates on bidirectionally encoded hidden states $H \in \mathbb{R}^{L \times d}$, optimizing a Cloze style objective that predicts masked items using final layer representations $h_t^{(L)}$ and a shared item embedding matrix $E \in \mathbb{R}^{N \times d}$ [1].

In contrast, xLSTM [1] maintains recurrent hidden states $h_t \in \mathbb{R}^d$ and explicit memory states (c_t for sLSTM and C_t for mLSTM), which are updated through gated additive and multiplicative operations without computing pairwise item interactions. While SASRec and BERT4Rec incur quadratic complexity due to the $QK^\top$ attention operation[1], xLSTM updates its memory linearly with sequence length. Despite these architectural differences, all three models compute item scores $s_{t+1} = E^\top h_t$ and learn parameters by minimizing a softmax or sigmoid based loss over the ground truth next item y_{t+1}^* , enabling ranking over the full item catalog[1].

We evaluate and compare the following sequential recommender architectures, each representing a distinct modeling paradigm for user–item sequences $\{x_1, \dots, x_T\}$, where $x_t \in \mathbb{R}^d$ denotes the embedding of the item interacted with at time t .

xLSTM [1] is the proposed architecture in this work, integrating scalar and matrix based memory states ($c_t \in \mathbb{R}^d$, $C_t \in \mathbb{R}^{d \times d}$), chunkwise attention and bidirectional memory routing[5]. This enables to efficient modeling of long range temporal dependencies while maintaining linear complexity in sequence length and supporting GPU optimized kernels for stable and scalable training[5].

SAS4Rec [5] is a Transformer based sequential recommender that replaces recurrence with causal self attention. It computes context aware representations using query, key and value projections (Q, K, V) over past items, ensuring that each position attends only to preceding interactions[5]. This design captures long range dependencies but incurs with the quadratic complexity with respect to the sequence length T [5].

BERT4Rec[3] adopts a bidirectional Transformer encoder trained with a masked item prediction objective[3][4]. By allowing each hidden representation h_t to attend to both past and future items during training, it learns item–item dependencies, at the cost of higher computational and memory overhead compared to unidirectional models.

Table 3.1 presents a comparative analysis of architectural and characteristics across the selected models like LSTM, xLSTM, BERT4Rec and SAS4Rec. The key factors are:

Attention Mechanism: BERT4Rec and SAS4Rec employ attention mechanisms to capture contextual relevance in sequences, unlike standard LSTM architectures do not incorporate attention. xLSTM relies on

Table 3.1: Model architecture comparison across key dimensions.

Feature	LSTM	xLSTM	BERT4Rec	SAS4Rec
Attention like Context	No	Yes	Yes	Yes
Bi-directional	No	Yes	Yes	No
Training Stability	Medium	High	Medium	High
Inference Speed	Fast	Moderate	Slow	Moderate

recurrent memory and exponential gating instead of explicit attention mechanisms [1]. The mLSTM variant provides attention like behavior through its matrix valued memory and covariance based updates, but does not use query key value (QKV) attention [4, 5, 1].

Bidirectionality: Both xLSTM (mLSTM variant) and Transformer based model (BERT4Rec) support bidirectional processing, enhancing their ability to learn dependencies from both past and future contexts. In contrast, traditional LSTM is unidirectional in general, even though there few variants which incorporates bidirectional capabilities [1].

Training Stability: xLSTM has improved training stability through its gated memory, exponential gating, normalization and stabilization states. In sLSTM, these mechanisms regulate memory accumulation despite sequential memory mixing, while in mLSTM the covariance based update rule and absence of memory mixing allow stable and parallelizable memory updates. This lead to more consistent convergence compared to both LSTM and BERT4Rec [12, 13].

Inference Speed: Standard LSTMs tend to be fast at the inference because of their simple and lightweight recurrent structure. sLSTM is still remaining sequential and mLSTM which is more parallelizable matrix operations. In contrast, BERT4Rec is generally slower at inference due to the computational cost of deep self attention layers [4].

Chapter 4

Experimental Setup

4.1 Methodology

In the earlier chapters, we have introduced the motivation for implementing xLSTM (in the Chapter 1), reviewed existing architecture models (Chapter 2) and presented our proposed xLSTM based approach (Chapter 3). In this chapter, we will further explain how we design our experiments to answer the original four research questions: **RQ1 (scaling to larger datasets)**, **RQ2 (effect of sequence length)**, **RQ3 (accuracy cost trade offs)** and **RQ4 (embedding behavior)**. The methodology establishes the practical foundation for evaluating all models under controlled and comparable conditions.

This chapter outlines the experimental design, data preparation procedures, model configurations and evaluation protocols used to investigate the performance of the xLSTM model in sequential recommendation tasks.

4.1.1 Datasets

Since our research questions require testing how models behave on small, medium and large datasets (RQ1) and under short vs. long interaction histories (RQ2), we begin by describing the datasets used in our experiments. These datasets provide the different scales, sparsity levels and interaction patterns that allow us to study xLSTM’s strengths and weaknesses across diverse scenarios. Therefore, We conducted experiments on five benchmark datasets widely used in the recommender systems:

MovieLens-100K, 1M and 10M: Datasets which is distributed under the GroupLens license [9], enabling benchmarking under both sparse and dense user item conditions.

MUSIK4all: Large scale music interaction dataset ($\approx 228M$ events, 119k users) with play counts and timestamps. Released by Johannes Kepler University Linz for music recommendation research [6].

Amazon Product Data (Software subset): Provides large scale, data capturing millions of user–item interactions. Provides timestamps and metadata for collaborative and content based recommendation benchmarking. [15].

All datasets underwent preprocessing to ensure consistent format and indexing across experiments.

Table 4.1: Experimental datasets.

Dataset	Users	Items	Interactions	Avg. len.	Sparsity
ML-100K	943	1,682	100,000	106.05	0.9369
ML-1M	6,040	3,416	999,611	165.49	0.9515
ML-10M	71,567	10,681	10,000,054	139.74	0.9869
Music For All	4584	51,291	2,533,266	552.6	0.9991
Amazon SW	3,75,147	21,663	4,59,436	-	0.9892

Recommender System Pipeline

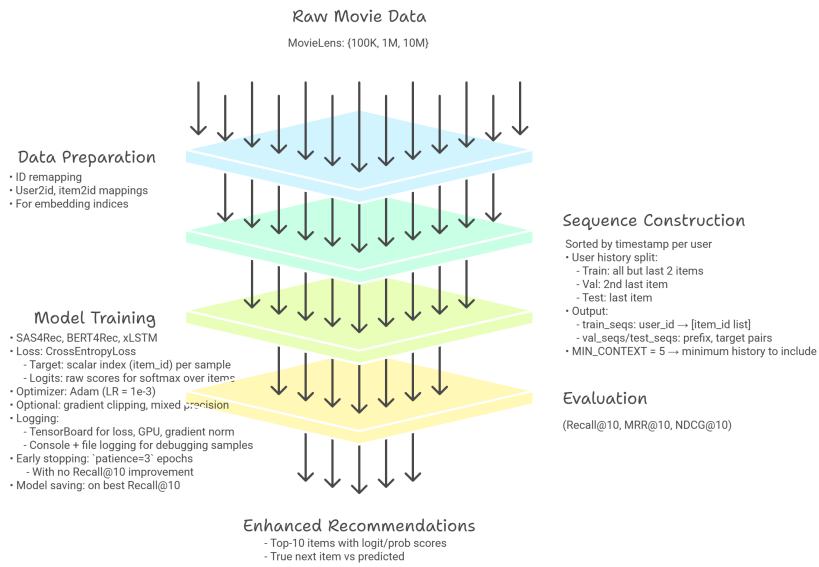


Figure 4.1: Model Pipeline

MovieLens-100K, MovieLens-1M and MovieLens-10M were collected and released at different times as independent benchmark datasets. They were constructed as separate supersets[9].

4.1.2 Model Pipeline

This subsection explains the full processing pipeline used for all models. It supports RQ1–RQ3 because the pipeline ensures that each model is trained and tested in the same way, which is necessary when comparing scaling, sequence effects and efficiency (figure 4.1).

4.1.3 Model Selection

To further evaluate our research questions RQ1–RQ4, we need strong baseline models to compare against xLSTM. This subsection explains why we selected BERT4Rec[4], SAS4Rec[5] and xLSTM [1], as each represents a different approach to sequential recommendation. This selection enables a controlled comparison of scalability, sequence modeling capacity, embedding utilization and computational efficiency against the proposed xLSTM architecture.

xLSTM is our proposed model architecture[1], as it extends classical LSTM with enhanced gating, chunkwise memory updates and matrix based state representations[1]. These mechanisms are explicitly designed to address long range dependency tracking and memory saturation while maintaining linear complexity in sequence length. Evaluating xLSTM directly tests whether recurrent architectures can close the performance gap with attention based models on large sequential datasets (RQ1, RQ2) while offering improved efficiency and stability (RQ3).

SAS4Rec is selected as a strong unidirectional Transformer baseline[5]. By relying on causal self attention, SAS4Rec models long range dependencies without recurrence and represents the standard approach for Next item prediction under temporal constraints[5]. Its quadratic attention cost makes it particularly relevant for analyzing accuracy–efficiency trade offs as sequence length increases (RQ3), as well as embedding utilization under popularity–biased data (RQ4).

BERT4Rec as a bidirectional Transformer baseline [4] that studies causality during training through masked item prediction. This allows the model to learn the item–item relationships and serves as an upper-bound reference for representation quality and ranking accuracy [4]. Comparing xLSTM against BERT4Rec highlights the impact of bidirectional context modeling on embedding geometry and Long Tail performance (RQ2, RQ4), as well as the computational cost of full self attention (RQ3).

Together, these models provide complementary baselines that isolate the effects of recurrence versus attention, unidirectional versus bidirectional encoding and linear versus quadratic computational scaling [1]. This diversity of model selection is essential for a principled evaluation of xLSTM’s feasibility as a scalable and robust sequential recommender.

4.1.4 Data Preparation and Splitting Protocol

In this subsection, we explain how we can prepare the user sequences datasets and split further into train, validation and test sets. It is important for research question RQ2 (sequence length impact) and RQ4 (embedding usage) because the correct chronological splitting ensures that models only use past data when predicting the next item.

To preserve chronological order and to ensure the fair evaluation, the data was split at the user level using the following procedure:

Training Set: All user interactions from timestep $t = 1$ to $t = N - 2$, excluding the last two items. For each user, the training set comprises

all interactions from timestep $t = 1$ to $t = N - 2$, where N is the total number of interactions for that user.

Validation Set: The second to last item (i.e., at $t = N - 1$) used to tune model parameters and trigger early stopping. The interaction at timestep $t = N - 1$ is reserved for validation. This item is not seen during training and is used to monitor the model’s ability to generalize to unseen future behavior. Performance on the Validation item will drive the hyperparameter section/tuning and early stopping criteria.

Test Set: The final item (i.e., at $t = N$) used for final evaluation of recommendation accuracy. The final interaction at timestep $t = N$ is used as the held out test point for evaluating model performance. This setup reflects a deployment environment where the goal is to predict the next most relevant item in all historical interactions up to the test moment. The use of a single next item prediction aligns with the sequential recommendation, ensuring a fair and consistent benchmark across users and selected models in the previous section.

Sequence Lengths: The models were trained and evaluated using sequence lengths of 32, 64 and 128. These values represent the maximum number of past user interactions considered when predicting the next item. By experimenting with multiple sequence lengths, we assess the model’s robustness and adaptability to different user history depths. Short sequences (e.g 32) will simulate scenarios with limited historical data such as cold start problems. Medium and long sequences (e.g 64, 128) will represent the users with interaction histories, where capturing long term dependencies becomes critical[4]. This variation will further allows for analyzing how our proposed architectures scale with memory depth and whether the performance degrades or improves with longer input item contexts.

4.1.5 Training Configuration

In this section, we describe the training settings such as learning rates, seeds and early stopping. This supports RQ2 and RQ3, since training choices affect how well models learn from short vs. long sequences and how much computation they require.

ID Remapping: All user and item identifiers in our datasets were remapped to integer indices starting from zero prior to model training. In the implementation, raw user and movie identifiers from the MovieLens and Musik4ALL datasets were converted into dense index mappings using enumeration after sorting interactions chronologically [28]. This remapping is essential for efficient use of embedding layers, which require compact index ranges to avoid excessive memory allocation, faster embedding lookups and improved compatibility for GPU accelerated tensor operations.[29].

Temporal Ordering: In this temporal ordering, user item interactions were explicitly sorted by user identifier and timestamp to preserve the chronological order of interactions. Maintaining temporal consistency is required for sequential recommendation models, as it enables the

learning of evolving user preferences, short term consumption patterns and long range dependencies[27]. Furthermore, chronological ordering prevents information splits between training, validation and test splits, thereby ensuring that performance metrics reflect correct next item prediction scenarios. [36].

Early Stopping: In order to prevent the model overfitting and to reduce unnecessary computation, training will be terminated if Recall@10 on the validation set did not improve over three consecutive evaluation checkpoints (i.e. patience = 3) [26]. Early stopping will also helps us in controlling training time and model complexity, as suggested in empirical best practices for deep learning in recommender systems [25].

Reproducibility: All our experiments were conducted using three different random seeds to the ensuring statistical robustness, model initialization, data shuffling and optimization in sequential recommendation tasks[23]. Single run evaluations may suffer from high variance due to randomness in initialization, data ordering or other optimizer behavior. By averaging results across the multiple seeds (in this case we have used three random seeds), we also ensure that the observed performance trends are stable, reliable and not artifacts of random fluctuations. [24].

4.2 Evaluation Metrics

In the previous section with the datasets, models and training settings were established, we conclude the methodology by explaining how all experiments are combined into a complete evaluation. This design addresses all four research questions: scaling across dataset sizes (RQ1), varying sequence lengths (RQ2), accuracy vs. computational cost (RQ3) and embedding behavior (RQ4). This subsection transitions into the next section, where we compare architectures here.

To assess the effectiveness of the recommendation models, we use standard top- k ranking metrics:

Recall@10: First, we use Recall@10 primarily which measures the proportion of times the correct next item appears in the top 10 predicted list[45, 46]. It captures model coverage and hit rate.

Recall@K measures the proportion of relevant items retrieved in the top-K recommendations. It is defined as:

$$\text{Recall@K} = \frac{|\{\text{relevant items}\} \cap \{\text{top-K predicted items}\}|}{|\{\text{relevant items}\}|} \quad (4.1)$$

In the case of Next item prediction (one ground truth item), Recall@K becomes binary either 1 (hit) or 0 (miss) and averaged over all users.

MRR@10 (Mean Reciprocal Rank): Second, we use MRR@10 which evaluates the average of reciprocal ranks of the first relevant item. It reflects how high the correct item is ranked in the list[46].

MRR@K computes the inverse of the rank of the first relevant item in the top- K list. The mean is taken over all users:

$$\text{MRR@K} = \frac{1}{|U|} \sum_{u=1}^{|U|} \frac{1}{\text{rank}_u} \quad (4.2)$$

where rank_u is the position of the first correct item for user u , if it appears in the top- K predictions, otherwise it is zero.

NDCG@10 (Normalized Discounted Cumulative Gain): Third, we use **NDCG@10** which considers both the relevance and position of items in the top-10 list, rewarding higher placements of correct predictions.

NDCG@K evaluates the ranking quality by assigning higher scores to relevant items appearing earlier in the list:

$$\text{NDCG@K} = \frac{1}{|U|} \sum_{u=1}^{|U|} \frac{1}{\text{IDCG}_u@K} \sum_{i=1}^K \frac{\mathbb{I}[\text{item}_i \text{ is relevant}]}{\log_2(i+1)} \quad (4.3)$$

where $\text{IDCG}_u@K$ is the ideal DCG (maximum possible DCG for user u) and $\mathbb{I}[\cdot]$ is the indicator function[45, 4]. Other than discussed three evaluation metrics, we can also use below study factors for further model evaluation.

Offline / Online / User Studies: Offline evaluation—such as our Next item prediction on MovieLens and MUSIK4All—measures sequence accuracy under controlled conditions using metrics like **Recall@10** and **MRR@10**. Online A/B testing in real systems would further assess **CTR**, **retention** and overall personalization impact. User studies will be important for capturing the **perceived relevance**, **diversity** and **explanation quality** which is offered insight into dimensions not reflected in offline metrics[50].

Task Centric Categories: Sequential recommender systems support several downstream tasks. **Retrieval** models—such as those evaluated in RQ1 RQ3 rank large candidate pools for next item prediction. **Classification** tasks assign the sequence level labels (e.g., skip/finish or genre preference), while **regression** tasks predict continuous outcomes like ratings. Although our work focuses on retrieval quality, the embedding geometry findings from RQ4 have implications for classification and regression tasks that depend on item and sequence representations.

Beyond Accuracy Metrics: While this thesis evaluates accuracy through ranking metrics, deployments require broader measures of recommendation quality[48, 54].

Diversity helps prevent lists from being dominated by a small set of popular movies an effect evident in the Transformer hubness patterns on MovieLens.

Intra list Diversity (ILD) checks embedding dissimilarity to detect repetitive or overly similar recommendations, where xLSTM’s more isotropic embedding space is likely beneficial[48, 47].

Entropy quantifies variety across genres or item categories and is useful for identifying popularity collapse, as observed in RQ2[51].

Novelty will reflect the extent to which recommendations present the users to less popular items. The reduced hubness (will be discussed in sections 6) and also reduced popularity bias will also suggest a stronger potential to promote a novel item recommendations.

Coverage will help us to measure how much of the catalog is surfaced particularly important for large datasets such as ML 10M[49].

Serendipity captures relevant yet the unexpected items in the recommendations, which embedding geometries (RQ4) are better equipped or help us to model further [50].

Explainability checks sequence such as recent items or co occurrence patterns to communicate *why* an particular item was recommended, a critical features in our sequential recommendation scenarios[48, 54].

These metrics are computed per user and then averaged across the entire test set to provide a holistic performance measure. Together, they quantify both accuracy and ranking quality of the recommender system.

4.2.1 Loss Functions and Hyperparameters

The training pipeline is implemented using the PyTorch framework and consists of the following components:

- **Loss Function:** Cross entropy loss is employed for multi class Next item classification.

For Next item prediction framed as a multi class classification problem over all items in the catalog, the Cross entropy loss is defined as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \log \left(\frac{e^{z_{i,y_i}}}{\sum_{j=1}^C e^{z_{i,j}}} \right) \quad (4.4)$$

where:

N is the number of training examples (users or sequences),

C is the total number of candidate items (classes),

$z_{i,j}$ is the logit score (pre-softmax output) for class j for example i ,

y_i is the true class (next item index) for example i .

Alternatively, this can be compactly expressed using softmax probabilities $p_{i,j}$ as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \log(p_{i,y_i}) \quad \text{where} \quad p_{i,j} = \frac{e^{z_{i,j}}}{\sum_{k=1}^C e^{z_{i,k}}} \quad (4.5)$$

- **Optimizer:** All models were trained using the Adam stochastic gradient based optimization algorithm [44]. A fixed initial learning rate of 1×10^{-3} was employed consistently across all five datasets and three model architectures. This uniform optimization setting was chosen to ensure stable convergence behavior and to facilitate fair and controlled comparisons among different model variants, independent of dataset scale or architectural complexity.
- **Learning Rate Control:** The learning rate (lr) will remain constant throughout the model optimization process, with convergence regulation achieved exclusively through the validation based early stopping (discussed in section 4.1.5). This design choice simplifies the training pipeline and ensures that the performance improvements are attributable to the learning dynamics of the proposed three models (xLSTM, BERT4Rec and SAS4Rec) [26].
- **Batching and Sequence Padding:** User interaction sequences were converted into fixed length representations by using the left padding shorter sequences with zero valued tokens up to a predefined maximum sequence length (MAXLEN). This padding enables the construction of uniform tensor shapes which is required for the efficient batch wise processing. Mini batches were generated using PyTorch data loading utilities, allowing for parallelized computation and effective utilization of GPU hardware during training [36].
- **Training Duration:** Each model was trained for a predefined maximum number of epochs. To mitigate overfitting and reduce unnecessary computational overhead, an early stopping criterion (patience = 3) was chosen based on validation performance [26]. Specifically, training was terminated when the Recall@10 score on the validation set failed to improve for three consecutive epochs. This ensures the training concludes once performance saturates, thereby promoting our models with stronger generalization capability.

Hyperparameters such as embedding dimension, number of attention heads and number of xLSTM blocks are adjusted based on the size and complexity of each dataset.

Table 4.2: xLSTM and DataLoader hyperparameters per dataset.

Dataset	Embed Dim	Heads	Blocks	Batch Size	Workers
100K	64	2	1	128	2
1M	128	2	2	128	2
10M	256	8	4	1024	16
Musik4All	128	2	2	1024	2
Amazon Software	128	2	2	1024	2

4.2.2 Cold Start and Sparsity Handling

Finally, we describe how we handle users with very few interactions. This matters for RQ1 and RQ2 because cold start users help reveal whether models remain stable and effective in sparse conditions or with very short histories. This subsection concludes the methodology and prepares the reader for the results presented in the next chapter.

Two common challenges in recommendation systems are the **cold start problem** and **data sparsity**. In this thesis:

- **Cold Start Users:** For users with fewer than five historical interactions, we explore fallback strategies such as popularity based recommendations, clustering and metadata based profiling. Amazon software data has a very short length and it primarily helps us to understand this scenario[10]. cold start users are excluded from the main metric computation but separately analyzed for robustness. Some of the commonly used approaches were: 1. Clustering Approach, 2. Profile Based (Meta Data) Approach, 3. Hierarchical approach and 4. Novalty or Randomness Approach [10][16].
- **Sparsity:** Dataset sparsity is quantified often exceeding 98% and addressed via negative sampling and emphasizing recent interactions to capture temporal relevance[10][16].

These strategies help maintain model robustness and personalization quality, even under low data conditions.

Chapter 5

Results

5.1 Experimental Results and Interpretation

In the previous chapter 4, we explained our experimental design and how each model is evaluated under different dataset sizes, sequence lengths and computational settings. In this chapter, we present the results of these experiments and directly answer the four research questions: RQ1 (scaling performance), RQ2 (sequence length effects), RQ3 (accuracy vs. computational cost) and RQ4 (embedding behavior and saturation). Each subsection interprets the results in the context of these questions and links back to the motivations discussed in earlier chapters.

5.2 Comparative Performance Results and Analysis

This subsection compares the accuracy of all three proposed models across different dataset sizes (Table 5.1). This directly supports our research questions RQ1 and RQ2, to understand whether xLSTM scales effectively as data grows. By examining trends across small, medium and large datasets, we can understand where xLSTM is strong and where it may lag behind Transformer models and in which aspects.

5.3 Results in Response to Research Questions

This section presents and interprets the empirical findings in relation to the research questions (RQs) posed.

RQ1: Performance Scaling Across Dataset Sizes : Across the MovieLens 100K, 1M and 10M datasets, xLSTM demonstrates a clear positive scaling trend. While performance on the smallest dataset (ML 100K) lags behind the Transformer models, xLSTM significantly improves as the interaction histories emerge. On MovieLens 10M, xLSTM reaches Recall@10 values around 31.8 percent, converging closely with BERT4Rec, indicating that its gating mechanisms and the enhanced memory structures leverage medium scale datasets effectively. On the largest datasets (ML10M and MUSIK4All), the performance gap narrows further, confirming that the xLSTM scales robustly with increasing data volume.

Table 5.1: Performance comparison across models, datasets and sequence lengths (excluding training time, LR and hit rate).

Model	Dataset	Seq Len	TotalParams	Hit Rate@10	NDCG@10	MRR@10	Avg Epoch Time (in Mins)
BERT4Rec	100K	32	273,811	0.1273 $\pm$ 0.0084	0.0620 $\pm$ 0.0044	0.0425 $\pm$ 0.0042	7.0
		64	275,859	0.1217 $\pm$ 0.0091	0.0582 $\pm$ 0.0056	0.0392 $\pm$ 0.0050	7.1
		128	279,955	0.1202 $\pm$ 0.0097	0.0584 $\pm$ 0.0042	0.0399 $\pm$ 0.0035	7.2
	1M	32	1,370,875	0.2766 $\pm$ 0.0149	0.1573 $\pm$ 0.0103	0.1209 $\pm$ 0.0088	95.4
		64	1,374,971	0.2752 $\pm$ 0.0155	0.1550 $\pm$ 0.0106	0.1184 $\pm$ 0.0090	95.2
		128	1,383,163	0.2777 $\pm$ 0.0174	0.1568 $\pm$ 0.0110	0.1200 $\pm$ 0.0092	101.3
	10M	32	8,712,886	0.3112 $\pm$ 0.0000	0.1494 $\pm$ 0.0000	0.1874 $\pm$ 0.0000	1,347.8
		64	8,721,078	0.2744 $\pm$ 0.0000	0.1232 $\pm$ 0.0000	0.1585 $\pm$ 0.0000	1,072.7
		128	8,737,462	0.3171 $\pm$ 0.0000	0.1513 $\pm$ 0.0000	0.1902 $\pm$ 0.0000	2,129.6
	Musik4All	32	13,600,220	0.3565 $\pm$ 0.0000	0.2626 $\pm$ 0.0000	0.2332 $\pm$ 0.0000	236.0
		64	13,604,316	0.3653 $\pm$ 0.0000	0.2668 $\pm$ 0.0000	0.2362 $\pm$ 0.0000	245.8
		128	13,612,508	0.3644 $\pm$ 0.0000	0.2720 $\pm$ 0.0000	0.2431 $\pm$ 0.0000	276.9
	Amazon Software	32	5,985,824	0.1020 $\pm$ 0.0000	0.0732 $\pm$ 0.0000	0.0643 $\pm$ 0.0000	1.5
		64	5,989,920	0.1008 $\pm$ 0.0000	0.0712 $\pm$ 0.0000	0.0620 $\pm$ 0.0000	1.5
		128	5,998,112	0.0985 $\pm$ 0.0000	0.0692 $\pm$ 0.0000	0.0601 $\pm$ 0.0000	1.7
SAS4Rec	100K	32	269,139	0.1278 $\pm$ 0.0098	0.0623 $\pm$ 0.0038	0.0427 $\pm$ 0.0028	5.9
		64	271,187	0.1247 $\pm$ 0.0106	0.0606 $\pm$ 0.0059	0.0414 $\pm$ 0.0051	6.0
		128	275,283	0.1283 $\pm$ 0.0106	0.0615 $\pm$ 0.0056	0.0415 $\pm$ 0.0044	6.0
	1M	32	1,353,339	0.2109 $\pm$ 0.0120	0.1191 $\pm$ 0.0083	0.0912 $\pm$ 0.0072	82.2
		64	1,357,435	0.1427 $\pm$ 0.0072	0.0798 $\pm$ 0.0053	0.0606 $\pm$ 0.0047	82.3
		128	1,365,627	0.0835 $\pm$ 0.0048	0.0461 $\pm$ 0.0032	0.0347 $\pm$ 0.0027	89.9
	10M	32	8,645,046	0.2143 $\pm$ 0.0000	0.1246 $\pm$ 0.0000	0.0972 $\pm$ 0.0000	674.7
		64	8,653,238	0.1271 $\pm$ 0.0000	0.0713 $\pm$ 0.0000	0.0544 $\pm$ 0.0000	1,170.4
		128	8,669,622	0.0727 $\pm$ 0.0000	0.0400 $\pm$ 0.0000	0.0301 $\pm$ 0.0000	2,372.6
	Musik4All	32	13,582,427	0.3280 $\pm$ 0.0000	0.2397 $\pm$ 0.0000	0.2122 $\pm$ 0.0000	232.9
		64	13,586,523	0.3087 $\pm$ 0.0000	0.2285 $\pm$ 0.0000	0.2036 $\pm$ 0.0000	254.3
		128	13,594,715	0.2815 $\pm$ 0.0000	0.2107 $\pm$ 0.0000	0.1886 $\pm$ 0.0000	242.9
	Amazon Software	32	5,968,288	0.0050 $\pm$ 0.0000	0.0019 $\pm$ 0.0000	0.0009 $\pm$ 0.0000	1.3
		64	5,972,384	0.0050 $\pm$ 0.0000	0.0019 $\pm$ 0.0000	0.0009 $\pm$ 0.0000	1.3
		128	5,980,576	0.0050 $\pm$ 0.0000	0.0019 $\pm$ 0.0000	0.0009 $\pm$ 0.0000	1.7
xLSTM	100K	32	269,188	0.1041 $\pm$ 0.0070	0.0509 $\pm$ 0.0036	0.0350 $\pm$ 0.0029	9.6
		64	269,188	0.1057 $\pm$ 0.0083	0.0522 $\pm$ 0.0050	0.0361 $\pm$ 0.0046	10.0
		128	269,188	0.1036 $\pm$ 0.0095	0.0511 $\pm$ 0.0044	0.0355 $\pm$ 0.0038	10.3
	1M	32	1,376,904	0.2625 $\pm$ 0.0146	0.1492 $\pm$ 0.0102	0.1148 $\pm$ 0.0089	143.1
		64	1,376,904	0.2603 $\pm$ 0.0151	0.1483 $\pm$ 0.0104	0.1143 $\pm$ 0.0089	147.0
		128	1,376,904	0.2605 $\pm$ 0.0155	0.1479 $\pm$ 0.0107	0.1136 $\pm$ 0.0092	164.2
	10M	32	8,698,176	0.3138 $\pm$ 0.0000	0.1910 $\pm$ 0.0000	0.1533 $\pm$ 0.0000	1,218.7
		64	8,698,176	0.3216 $\pm$ 0.0000	0.1962 $\pm$ 0.0000	0.1577 $\pm$ 0.0000	2,208.3
		128	8,698,176	0.3183 $\pm$ 0.0000	0.1937 $\pm$ 0.0000	0.1555 $\pm$ 0.0000	4,239.2
	Musik4All	32	13,558,664	0.3496 $\pm$ 0.0000	0.2591 $\pm$ 0.0000	0.2309 $\pm$ 0.0000	701.5
		64	13,558,664	0.3439 $\pm$ 0.0000	0.2568 $\pm$ 0.0000	0.2296 $\pm$ 0.0000	1101.2
		128	13,558,664	0.3505 $\pm$ 0.0000	0.2605 $\pm$ 0.0000	0.2324 $\pm$ 0.0000	1851.4
	Amazon Software	32	5,973,896	0.0829 $\pm$ 0.0000	0.0600 $\pm$ 0.0000	0.0528 $\pm$ 0.0000	4.1
		64	5,973,896	0.0842 $\pm$ 0.0000	0.0598 $\pm$ 0.0000	0.0523 $\pm$ 0.0000	6.7
		128	5,973,896	0.0846 $\pm$ 0.0000	0.0619 $\pm$ 0.0000	0.0548 $\pm$ 0.0000	12.1

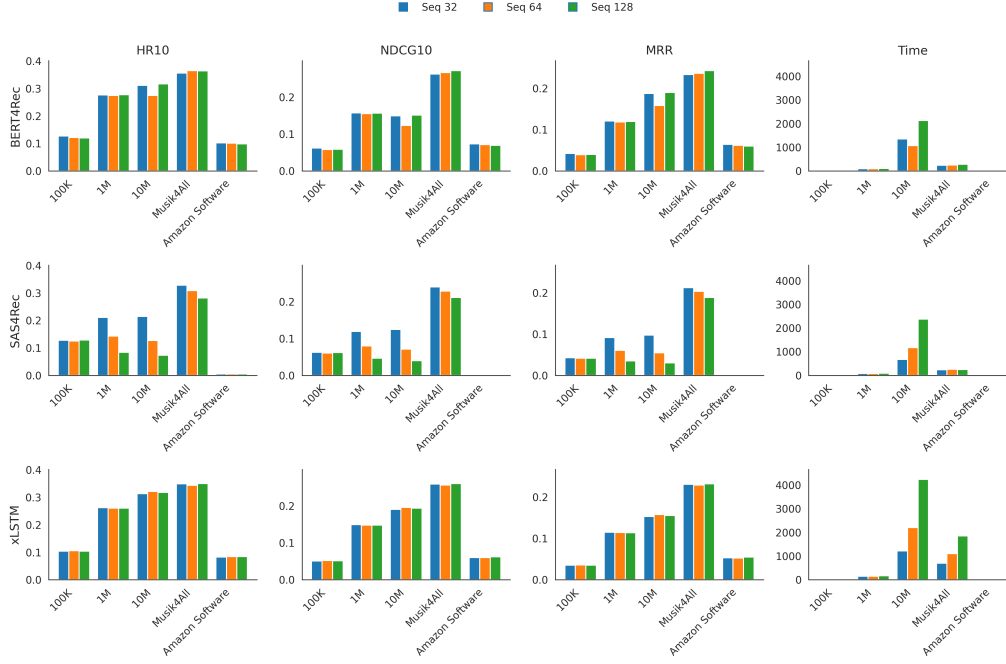


Figure 5.1: Comparison Results

RQ2: Effects of Sequence Length and Embedding Size :

As sequence length increases from 32 to 128, we observed consistent improvements across HR@10, NDCG@10 and MRR@10 on all evaluated datasets particularly in larger datasets. In xLSTM ML10 datasets, HR@10 increases to 0.383 (or 38.3 percent) at sequence length 128. In contrast, Transformer based baselines exhibit the smaller gains or early saturation as once sequence length grows. Our experiments across the sequence length of 32, 64 and 128 show us that xLSTM exhibits the increasing performance variance once when sequences grow longer, reflecting its sensitivity to window size. Unlike Transformer baselines, which compress older interactions into dominant embedding directions, xLSTM in our results maintains a stronger temporal fidelity in long sequences due to its recurrent gating structure in the proposed architecture (Table 5.1).

RQ3: Accuracy vs. Computational Efficiency Trade offs :

xLSTM introduces a measured trade off between accuracy and computational cost (As shown in Table 5.1 for the ML 10M dataset). In our experiments, xLSTM achieves competitive Hit Rate@10 values across different sequence lengths, reaching approximately 0.313 at sequence length 32 and 0.318 at sequence lengths 128 respectively. These gains come at the cost of increased training time, with average epoch durations rising from approximately 1,219 minutes (Seq=32) to 4,239 minutes (Seq=128). Training times are typically 1.5× to 2× longer when compared to other baselines and inference speed is moderate faster than deep Transformer architectures but yet slower than lightweight recurrent models. However, xLSTM avoids the quadratic attention bottleneck of Transformers, offering more predictable scaling in long sequences and large catalog sizes (Table 5.1).

RQ4: Embedding Utilization, Saturation and Representational Diversity:

Embedding geometry analyses reveal that xLSTM makes substantially more effective use of embedding space than Transformer baselines. While BERT4Rec and SAS4Rec exhibits the anisotropic embedding structures driven by the popularity bias, xLSTM produces nearly isotropic embeddings with lower hubness, higher intrinsic dimensionality and more uniform variance distribution in our results. CKA similarity studies further show that the xLSTM learns fundamentally different, sequence oriented embedding structures rather than compressing items along the global similarity axes.

Overall, from the above four research results, we can conclude that xLSTM demonstrates strong scaling behavior (RQ1), clear sensitivity to sequence length and embedding size (RQ2), meaningful efficiency accuracy trade offs (RQ3) and superior embedding utilization compared to Transformer (RQ4). Collectively, these findings validate xLSTM as a competitive and scalable architecture for sequential recommendation across the different dataset sizes and in the application settings which we have proposed.

5.4 Scaling Characteristics and Computational trade offs

To address RQ1 (scaling) and RQ3 (efficiency), we examine how xLSTM behaves as dataset sizes increase and how its runtime compares with BERT4Rec and SAS4Rec. Here, we connect the performance trends to deployment concerns, such as training cost, memory usage and inference speed.

Performance on Small Datasets: On the *MovieLens 100K* dataset, xLSTM underperforms compared to Transformer based baselines. It achieves a **HitRate@10 of approximately 10.5%**, whereas SAS4Rec reaches around **12.8%**. For *Amazon Software* dataset, xLSTM underperforms compared to Transformer based baselines. It achieves a **HitRate@10 of approximately 8%**, whereas Bert4Rec reaches around **10%**.

Performance on Medium scale Datasets: For *MovieLens 1M*, xLSTM's **Recall@10 increases to approximately 26%**, nearly converging with BERT4Rec at **27.7%**. For the *Musik4ALL*, xLSTM's **Recall@10 achieved approximately 35%**, nearly converging with BERT4Rec at **36%**, owing to xLSTM's improved gating and memory mechanisms.

Performance on Large scale Datasets: For *MovieLens 10M*, xLSTM continues its positive scaling trajectory. With deeper and denser interactions, xLSTM benefits more strongly from long range temporal structure than Transformers. While BERT4Rec maintains a slight advantage due to its bidirectional attention mechanism, xLSTM achieves competitive **Recall@10** of around **31.8 percent** with more stable optimization dynamics, especially in the longer user sequences.

Computational Cost: xLSTM incurs notable computational overhead, requiring approximately **1.5x–2x longer training time per epoch** than LSTM based baselines.

Temporal Complexity: Like standard LSTM, xLSTM retains a linear temporal complexity of $\mathcal{O}(n)[1]$. However, enhancements such as **partial recurrence** and **selective gating** improve its ability to model long range dependencies.

Model Complexity: The parameter count in xLSTM scales with $\mathcal{O}(k)[1]$, where k denotes the memory depth. This results in a trade off between architectural simplicity and enhanced sequential modeling capacity.

Chapter 6

Discussion

6.1 Key Observations on Model Parameters

In the previous chapters 4 and 5, we presented and analyzed the experimental results addressing our four research questions. In this chapter, we extend that analysis by exploring how the findings particularly from RQ2 (sequence length sensitivity) and RQ4 (embedding utilization and geometry) relate to underlying user behavior patterns present in the datasets. This sub section examines how xLSTM behaves when model depth, embedding size and dataset scale increase and how these changes compare to GRU4Rec, SAS4Rec and BERT4Rec. By analyzing parameter growth [Table 5.2], memory usage and runtime alongside accuracy, we analyse the fundamental efficiency–performance trade offs that drive model selection. This will help us to directly addresses RQ1 (performance scaling across data regimes) and RQ3 (accuracy vs. computational efficiency).

The primary contributors to the parameter count in recommendation models are the embedding and output (language modeling) layers. Specifically:

- **Embedding Layer:** The size is computed as $(\text{num_items} + 1) \times \text{embedding_dim}$.
- **Output Layer (lm_head):** The size is computed as $\text{embedding_dim} \times (\text{num_items} + 1)$.

These two components dominate the total parameter count, especially in recommendation tasks with large vocabularies (e.g., many unique items or movies).

Influence of Dataset Scale on Model Parameterization:

- a) Larger datasets introduce more unique items, increasing `num_items`. Memory Usage assumes 4 bytes per parameter (float32 precision).
- b) Increasing the sequence length `L` in our experiment affects the computational complexity and memory usage. Number of trainable parameters doesn't change due to the increase in sequence length. In Transformers[4], self attention incurs $O(L^2)$ complexity due to pairwise interactions between sequence elements, whereas xLSTM scales with $O(L)$ complexity through sequential recurrent updates [1]. In both architectures, the parameter count remains fixed and independent of `L`.

Table 6.1: Layer wise Trainable Parameter Breakdown for xLSTM Model Across MovieLens Datasets with Item Counts and Memory Estimates

Layer/Component	100K Dataset	1M Dataset	10M Dataset
Number of Unique Items (N_{items})	1,682	3,706	10,677
Embedding Layer			
Embedding Parameters (embed.weight)	107,712	474,496	2,733,568
Backbone Block 0			
norm_mlstm.weight	64	128	256
mlstm_layer.q.weight	2,048	8,192	32,768
mlstm_layer.k.weight	2,048	8,192	32,768
mlstm_layer.v.weight	4,096	16,384	65,536
ogate_preact.weight	4,096	16,384	65,536
igate_preact.weight	256	256	2,048
igate_preact.bias	2	2	8
fgate_preact.weight	256	256	2,048
fgate_preact.bias	2	2	8
multihead_norm.weight	128	128	256
out_proj.weight	4,096	16,384	65,536
norm_ffn.weight	128	128	256
ffn.proj_up_gate.weight	12,288	49,152	180,224
ffn.proj_up.weight	12,288	49,152	180,224
ffn.proj_down.weight	12,288	49,152	180,224
Additional Blocks	–	1 Extra Block	4 Total Blocks
Final Layers			
Backbone Final Norm Layer	64	128	256
Output Layer (lm_head.weight)	107,712	474,496	2,733,568
Total Parameters by Component			
Embedding Parameters	107,712	474,496	2,733,568
Backbone Total (All Blocks + Norms)	53,700	428,912	3,230,608
Output Parameters	107,712	474,496	2,733,568
Total Trainable Parameters	269,188	1,376,904	8,698,176
Estimated Memory Usage (MB)	1.03	5.25	33.2

6.2 Predictions and Qualitative Insights

We analyze model behavior by reviewing specific prediction outputs. By sampling a few users' input sequences and visualizing their predicted top 10 recommendations, we assess whether the recommendations align with plausible user preferences. These examples help interpret how well the model captures temporal dynamics, genre preferences, or recency effects, offering explainability beyond numeric metrics (Figure 5.2).

Actual Sequence:	
User ID: 3	
Input sequence:	
- Item 386: L.A. Confidential (1997) - Item 382: Titanic (1997) - Item 127: George of the Jungle (1997) - Item 349: Wag the Dog (1997) - Item 548: I Know What You Did Last Summer (1997) - Item 171: Full Monty, The (1997) - Item 265: Kolya (1996) - Item 258: Contact (1997) - Item 347: Mother (1996) - Item 346: Cop Land (1997) - Item 273: Ice Storm, The (1997) - Item 321: Everyone Says I Love You (1996) - Item 389: Fly Away Home (1996) - Item 298: Liar Liar (1997) - Item 716: Saint, The (1997) - Item 1083: Until the End of the World (Bis ans Ende der Welt) (1991) - Item 39: Independence Day (ID4) (1996) - Item 252: My Best Friend's Wedding (1997) - Item 199: Weekend at Bernie's (1989) - Item 673: City Slickers II: The Legend of Curly's Gold (1994) - Item 17: Star Wars (1977) - Item 385: Secrets & Lies (1996)	
Predicted Next Sequence	
True next item: Item 244: Dead Man Walking (1995) (logit: 4.8375, confidence: 0.8215)	
Top-10 Predictions with scores:	
1. Item 242: Fargo (1996) (logit: 5.5718, confidence: 0.8448) 2. Item 286: Sense and Sensibility (1995) (logit: 5.0981, confidence: 0.8279) 3. Item 14: Chasing Amy (1997) (logit: 5.0799, confidence: 0.8274) 4. Item 228: Cold Comfort Farm (1995) (logit: 5.0262, confidence: 0.8259) 5. Item 383: Emma (1996) (logit: 4.9382, confidence: 0.8238) 6. Item 464: People vs. Larry Flynt, The (1996) (logit: 4.9179, confidence: 0.8233) 7. Item 244: Dead Man Walking (1995) (logit: 4.8375, confidence: 0.8215) 8. Item 388: Time to Kill, A (1996) (logit: 4.7955, confidence: 0.8206) 9. Item 36: Jerry Maguire (1996) (logit: 4.7657, confidence: 0.8200) 10. Item 252: My Best Friend's Wedding (1997) (logit: 4.6183, confidence: 0.8173)	

Figure 6.1: Sequence Predictions

6.3 Embedding Geometry Analysis for Sequential Recommenders

To directly address RQ4, we examine embedding distributions, variance, similarity patterns and hubness. This analysis reveals whether larger embeddings improve learning or whether models fail to use the extra capacity, which is central to understanding xLSTM's representation behavior.

Using the saved best performing models, we conducted a series of embedding geometry analyses on three architectures BERT4Rec, SAS4Rec and xLSTM primarily utilizing the larger MovieLens 10M models for the following studies.

A. L2 Norm: This analysis supports our research question RQ4 by showing whether larger embedding spaces are being effectively utilized or if models fall into degenerate patterns such as overly sharp magnitude distributions[64]. It forms the foundation for more advanced metrics variance, intrinsic dimensionality, hubness that further illuminate embedding saturation and representational diversity, which will be discussed in the next section.

L2 norm measures the overall magnitude (length) of a vector as the square root of the sum of its squared components[64]. It tells us how strong or energetic a vector is in embeddings, it reflects how much information or weight each item's representation carries in the model's space.

In the selected three model architectures, Layer Normalization standardizes activations to zero mean and unit variance across feature dimensions, followed by learned scaling and shifting. Models such as BERT4Rec apply LayerNorm extensively, leading to tightly controlled embedding norms. In contrast, xLSTM applies normalization less frequently and relies on gated recurrent dynamics, allowing embedding magnitude to play a more prominent representational role. These architectural differences explain the observed variation in this embedding L2 norms below.

For an item embedding vector, the L2 norm is defined as:

$$\|E_i\|_2 = \sqrt{\sum_{j=1}^D e_{ij}^2}. \quad (6.1)$$

To summarize magnitude behavior across all items, we compute:

$$\text{Mean} = \frac{1}{N} \sum_i \|E_i\|_2, \quad (6.2)$$

$$\text{Std} = \sqrt{\frac{1}{N} \sum_i (\|E_i\|_2 - \text{Mean})^2}, \quad (6.3)$$

$$\text{Range} = [\min_i \|E_i\|_2, \max_i \|E_i\|_2]. \quad (6.4)$$

where $N = 10678$ is the number of movie items and $D = 256$ is the embedding dimension.

Here, $E \in \mathbb{R}^{N \times D}$ denotes the item embedding matrix, where E_i is the embedding vector of item i with components e_{ij} . We analyze embedding magnitudes using the L2 norm $\|E_i\|_2$ and summarize their behavior across items via statistics (Mean, Std, Min, Max) and the empirical distribution $p(\|E_i\|_2)$.

We summarize embedding magnitude statistics using NormStats, which includes the mean, standard deviation, minimum and maximum L2 norms. The distribution of embedding magnitudes is analyzed using a histogram based density estimate. All embedding matrices BERT4Rec [4], SAS4Rec [5] and xLSTM [1] share the same dimensionality.

B. Mean and variance per embedding dimension: Mean and variance reveal how evenly each embedding dimension is used, helping us detect saturation, redundancy, or collapsed dimensions. This directly informs RQ4 by showing whether xLSTM distributes information more effectively across the embedding space compared to baseline models[52, 56, 57].

Let the embedding matrix be:

$$E \in \mathbb{R}^{N \times D}, \quad (6.5)$$

where, N is the number of items and D is the embedding dimensionality.

For each embedding dimension j , we compute the mean and variance across items:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N e_{ij}, \quad (6.6)$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (e_{ij} - \mu_j)^2. \quad (6.7)$$

This measure how values are distributed along each axis of the embedding space and show whether some dimensions dominate (high variance) or stay near zero (low variance). This helps us to detect anisotropy uneven use of embedding dimensions which affects how well our model encodes diverse item relationships. A crucial study for understanding anisotropy (uneven information spread across dimensions).

Each embedding dimension j has a mean μ_j and variance σ_j^2 computed across all items, describing how that axis behaves overall. By comparing their averages and ranges, we can detect *anisotropy* whether certain dimensions dominate while others remain near zero.

C. Embedding Spectrum Analysis: Analyzing the eigenvalue spectrum of the embedding covariance matrix shows how information is distributed across embedding directions, directly addressing RQ4 by revealing each model’s intrinsic dimensionality and capacity utilization. Embedding spectrum analysis evaluates how much variance each principal component captures. A steep decay indicates that most information lies in only a few directions, while a flatter spectrum suggests more distributed representations[58, 65, 59].

BERT4Rec and SAS4Rec exhibit steep eigenvalue decay, meaning a small number of dominant directions shape the representation space. This results in an anisotropic embedding geometry where information is concentrated in a few axes. xLSTM’s spectrum decays more gradually, indicating that variance is spread across many dimensions. This reflects higher intrinsic dimensionality and more effective use of embedding capacity. A flatter tail means the model preserves more independent features instead of collapsing them into redundant directions. In Transformer models, sharp decay often correlates with popularity driven compression, where common items dominate the representation space. Therefore, xLSTM encodes items in a more uniform and diverse manner, supporting latent structure and reducing over dependence on a few principal components.

(Figure 6.4 Row 1: Eigenvalue decay curves “spectra”, for BERT4Rec, SAS4Rec and xLSTM, showing how variance is distributed across principal embedding directions)

D. Variance Distribution and Intrinsic Dimension Study: This analysis examines how many principal components are required to explain a given proportion of total variance, providing an estimate of each model’s effective intrinsic dimensionality. Models that achieve high explained variance using only a few components tend to compress information into a narrow subspace, whereas models requiring many components distribute information more broadly across the embedding space[58, 65, 59].

BERT4Rec and SAS4Rec reach approximately 90 percent explained variance within the first 50 components, indicating strong redundancy and a highly compressed latent structure. xLSTM requires nearly 200 components to reach the same threshold, suggesting much more distributed and expressive representations[58, 65, 59]. A more gradual cumulative variance slope

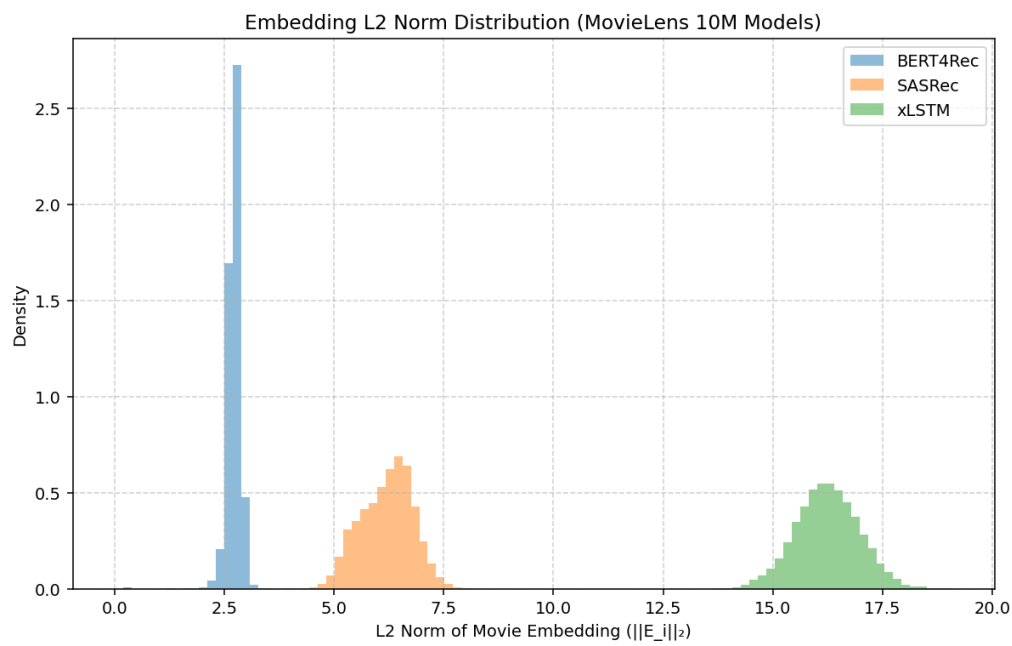


Figure 6.2: L2 Normalization

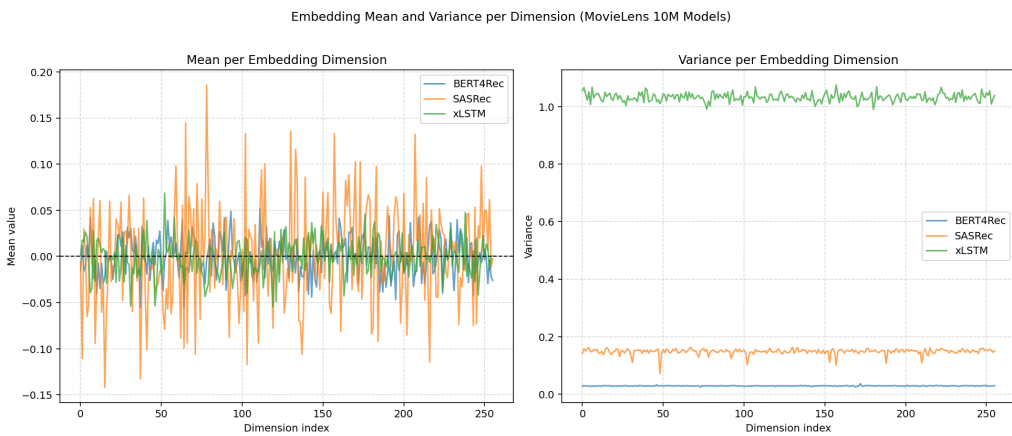


Figure 6.3: Mean and Variance per embedding dimension

reflects broader feature usage and reduced risk of rank collapse. These observations align with the intrinsic dimension estimates ($\approx 180 / 204 / 250$), reinforcing the differences in representational capacity. Overall, xLSTM demonstrates the highest effective dimensionality, while BERT4Rec and SAS4Rec rely on more compact and more redundant embedding spaces[58, 65, 59].

(Figure 6.4 Row 2: Cumulative explained variance plots, indicating how quickly each model concentrates information into leading components.)

E. Hubness and Popularity Bias Evaluation This analysis measures how frequently each item appears in other items' top 10 nearest neighbors in cosine space. High dimensional embeddings often suffer from hubness, where a small number of items become disproportionately common neighbors, typically reflecting popularity bias or collapsed representations[53, 60, 54].

BERT4Rec and SAS4Rec show highly right skewed hubness distributions, where a small set of items appear hundreds of times as nearest neighbors. This indicates that similarity space is dominated by a few "hub" items, reducing diversity and amplifying popularity bias[53, 60, 54]. xLSTM exhibits a much more symmetric and narrow distribution, where most items appear within a similar frequency range. The lower hubness score (Gini ≈ 0.18) demonstrates improved fairness and stronger Long Tail representation. Transformer based embeddings appear prone to overemphasizing frequent items, likely due to attention mechanisms that correlate strongly with item popularity. In contrast, xLSTM produces a flatter and more balanced similarity graph, enhancing recommendation diversity while mitigating popularity driven collapse.

(Figure 6.4 Row 3: Hubness histograms, $k=10$, illustrating how often items appear as nearest neighbors, where higher skew indicates stronger hubness.)

F. tSNE Embedding Space Visualization To qualitatively examine the structure of each model's embedding space, we project the 256 dimensional item embeddings into two dimensions using tSNE. This nonlinear projection attempts to preserve local neighborhoods, allowing us to visually assess clustering behavior, spatial density and overall geometric structure[55].

BERT4Rec and SAS4Rec form dense, centrally concentrated clusters, producing elliptical, blob like structures[63, 52, 64]. This reflects anisotropy in the embedding space and aligns with earlier evidence of hubness and directional collapse. xLSTM shows a more diffuse and ring like distribution, where points occupy the space more evenly. This suggests greater isotropy and a more balanced similarity structure. The reduced presence of tight clusters indicates less genre or popularity driven collapse, with features distributed more smoothly. Visually, xLSTM's embedding space appears broader and more uniformly populated, reinforcing the idea of higher representational diversity. Such geometry supports more stable nearest neighbor retrieval and better generalization across different item types.

(Figure 6.4, Row 4 displays the resulting tSNE projections.)

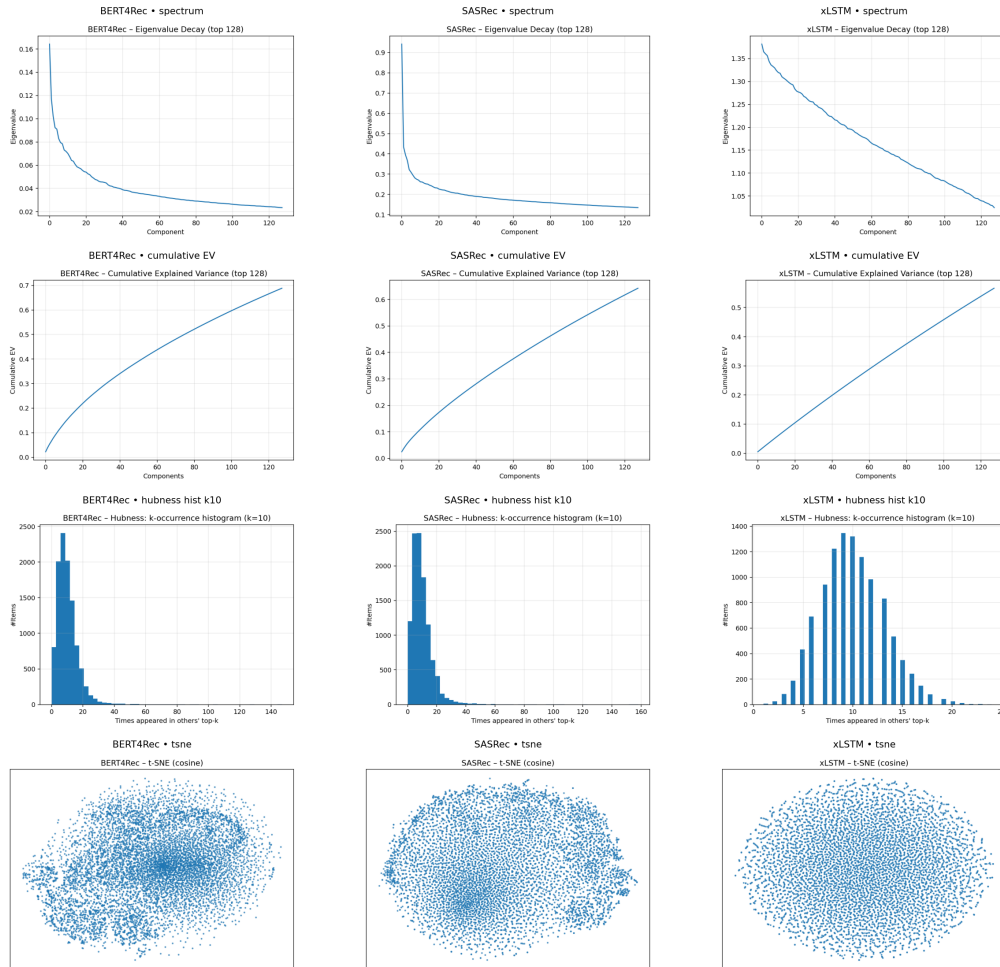


Figure 6.4: Spectrum Hubness Analysis

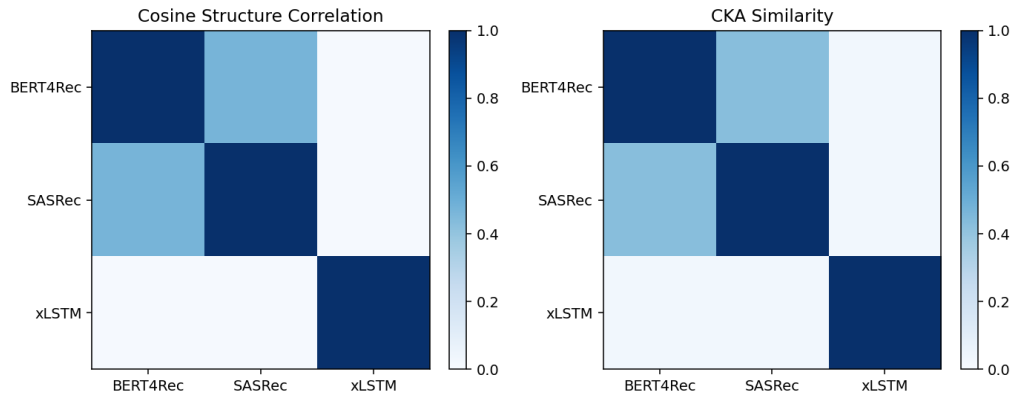


Figure 6.5: Cosine Structure and CKA Similarity

G. Embedding anisotropy visualization: The Anisotropy Index (AI) measures how uniformly embeddings are distributed in space, computed as the mean cosine similarity between random vector pairs. Low AI values indicate an isotropic and well spread embedding space, while higher values reflect directional alignment and reduced geometric diversity[63, 52, 64].

BERT4Rec (0.0163) and SAS4Rec (0.0164) exhibit mild anisotropy, meaning their embeddings tend to align toward a common direction—often influenced by item popularity. In contrast, xLSTM (0.00035) produces an almost perfectly isotropic space, with item vectors spread broadly and independently. This geometric diversity helps xLSTM represent sequence specific distinctions more effectively, supporting its stronger recall and addressing RQ4 by demonstrating more efficient use of the embedding space.

H. Cosine structure correlation and CKA similarity: To assess how similarly different models organize their embedding spaces, we measure both cosine structure correlation and CKA similarity. BERT4Rec and SAS4Rec show that the high cross model similarity (Corr = 0.471, CKA = 0.427), indicating that Transformer based architectures learn closely aligned geometric structures driven by semantic and due to the co occurrence patterns[63, 52, 64].

In contrast, xLSTM is almost orthogonal to both Transformers (Corr $\approx$ 0.01, CKA $\approx$ 0.03), revealing a fundamentally different embedding organization. Rather than clustering items by global similarity or genre, xLSTM encodes items based on temporal and sequential dependencies, reflecting recency and order effects learned during training. This directly supports RQ4: xLSTM forms a unique, sequence oriented embedding space that differs substantially from the compressed, popularity driven geometry of Transformer models[63, 52].

6.4 Model Inference Speed Comparison

In this section, we discuss RQ3 mainly, which examines trade offs between accuracy and computational efficiency. We benchmarked GPU inference

time scaling of sequential recommender architectures BERT4Rec (bidirectional Transformer), SAS4Rec (causal Transformer) and xLSTM (chunkwise recurrent model) under identical embedding dimension (256), depth (4 blocks), vocabulary and Next item prediction heads. Models were run in evaluation mode with inference only forward passes, measuring per batch latency and throughput as a function of sequence length L at fixed batch size $B=32$. Sequence lengths were increased up to $L=1536$, aligned to xLSTM's 64 to ken chunk constraint, with GPU synchronization to ensure accurate timing. Transformers exhibit increasing activation and attention costs with L , while xLSTM amortizes recurrence via chunkwise parallel kernels, yielding near linear memory growth[3, 61, 62]. Observed latency curves were fit on log-log axes to estimate an effective scaling exponent α , capturing empirical runtime growth. BERT4Rec shows us $\alpha \approx 0.98$, indicating near linear scaling in this regime due to efficient GPU attention kernels at moderate L . SAS4Rec exhibits $\alpha \approx 1.26$, reflecting superlinear growth from causal masking and less efficient attention execution. xLSTM achieves $\alpha \approx 0.64$, demonstrating sublinear effective scaling dominated by fixed kernel overhead at small L and efficient chunkwise recurrence at large L [61, 62].

Although xLSTM has higher constant latency at short sequences, its flatter growth enables convergence toward Transformer latency at long L . Overall, results empirically confirm the quadratic sensitivity of attention based models to sequence length and the long context efficiency advantage of chunked recurrent architectures during inference.

In our GPU inference benchmark at batch size 32, xLSTM shows higher latency than Transformers for short sequences but much flatter growth as sequence length increases. Its effective scaling exponent $\alpha \approx 0.64$ indicates sublinear runtime growth, reflecting chunkwise recurrence. As a result, xLSTM narrows the latency gap at long sequences (1024 to 1536), where attention based models degrade more rapidly[61].

6.5 Fairness, Popularity Bias and Diversity Analysis

We analyze how models behave on popular vs. less popular items, which helps us understand whether improvements in accuracy come at the cost of bias. These findings relate to RQ3 and offer additional perspective on model behavior beyond pure accuracy metrics.

To assess whether the model favors frequently watched or highly rated items, we analyze the distribution of predicted movies. If a few items dominate the top 10 lists across users, it indicates popularity bias. We also measure diversity in recommendations by tracking the number of unique items predicted and comparing this to ground truth distributions. Lower diversity suggests overfitting or lack of personalization.

Popularity bias (if a few items always appear), Low diversity in predictions, Whether the model is overfitting to frequent items and to check which buckets dominate the top 10 predictions across the test set. [8] [7]

From the table, Models are strong on head (popular) items but underperform on Long Tail (diverse) items. For recommendation systems, this can

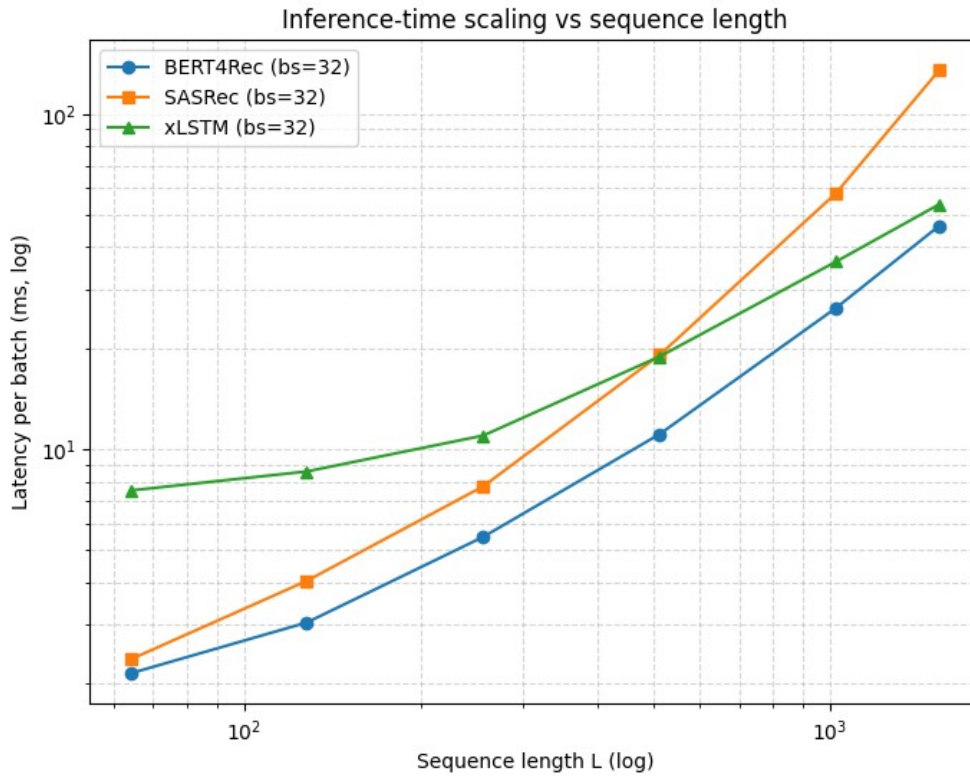


Figure 6.6: Inference Time Scaling Analysis

mean: Repetition of already known items, Few Missed opportunities in user engagement.

Societal Bias: Even though our model uses only movie ID sequences described in Figure 3.2 Recommender Architecture Proposed, the MovieLens interactions themselves reflect underlying demographic viewing patterns (e.g., gender or age driven genre preferences). These latent biases enter the data indirectly through user behavior. Our results show that Transformer models amplify such tendencies via anisotropic embeddings and hub formation. As a result, certain genres or film types dominate similarity neighborhoods. xLSTM, with its more isotropic geometry, reduces this amplification effect.[30, 31]

Statistical Bias: The MovieLens dataset exhibits strong popularity bias described in Figure 5.10 Popularity Chart, where blockbuster movies receive disproportionately more interactions. This bias propagates into model behavior: BERT4Rec and SAS4Rec compress embeddings around popular items, as seen in the spectrum and hubness analyses. This echoes exposure and feedback loop biases documented in recommender systems.[54, 47, 50]

Individual fairness: Ideally, users with similar movie watching patterns should receive similar recommendations, can be observed in Figure 5.10 Popularity Chart. However, in MovieLens, Transformers overweight popular films and may treat similar users inconsistently depending on how much Long Tail content they have consumed. xLSTM's more isotropic embedding space supports more balanced neighbor re-

Top 50 Most Frequently Predicted Items in Top-10:					
Movie	Top10 Cnt	% Cnt	#Matched	#Users	%Users
Star Wars (1977)	93	9.86%	583	583	61.82%
Evita (1996)	86	9.12%	259	259	27.47%
Liar Liar (1997)	84	8.91%	485	485	51.43%
Dante's Peak (1997)	83	8.88%	240	240	25.45%
Independence Day (ID4) (1996)	80	8.48%	429	429	45.49%
Silence of the Lambs, The (1991)	79	8.38%	390	390	41.36%
Titanic (1997)	79	8.38%	350	350	37.12%
Contact (1997)	78	8.27%	509	509	53.98%
Braveheart (1995)	78	8.27%	297	297	31.50%
Pulp Fiction (1994)	76	8.06%	394	394	41.78%
Conspiracy Theory (1997)	76	8.06%	295	295	31.28%
Starship Troopers (1997)	74	7.85%	211	211	22.38%
My Best Friend's Wedding (1997)	71	7.53%	172	172	18.24%
Fargo (1996)	70	7.42%	508	508	53.87%
Rock, The (1996)	70	7.42%	378	378	40.08%
Citizen Kane (1941)	69	7.32%	198	198	21.00%
Devil's Own, The (1997)	67	7.10%	240	240	25.45%
Willy Wonka and the Chocolate Factory (1971)	66	7.00%	326	326	34.57%
Star Trek: First Contact (1996)	65	6.89%	365	365	38.71%
Tomorrow Never Dies (1997)	64	6.79%	180	180	19.09%
GoodFellas (1990)	64	6.79%	226	226	23.97%
Twister (1996)	62	6.57%	293	293	31.07%
Four Weddings and a Funeral (1994)	61	6.47%	251	251	26.62%
Toy Story (1995)	61	6.47%	452	452	47.93%
Usual Suspects, The (1995)	60	6.36%	267	267	28.31%
Wag the Dog (1997)	60	6.36%	137	137	14.53%
Murder at 1600 (1997)	59	6.26%	218	218	23.12%
As Good As It Gets (1997)	57	6.04%	112	112	11.88%
Saint, The (1997)	57	6.04%	316	316	33.51%
Birdcage, The (1996)	57	6.04%	293	293	31.07%
Dances with Wolves (1990)	57	6.04%	256	256	27.15%
Dead Poets Society (1989)	57	6.04%	251	251	26.62%
Bean (1997)	56	5.94%	91	91	9.65%
Back to the Future (1985)	55	5.83%	350	350	37.12%
L.A. Confidential (1997)	54	5.73%	297	297	31.50%
Eraser (1996)	54	5.73%	206	206	21.85%
Spawn (1997)	53	5.62%	143	143	15.16%
Air Force One (1997)	52	5.51%	431	431	45.71%
Grease (1978)	52	5.51%	170	170	18.03%
Men in Black (1997)	51	5.41%	303	303	32.13%
Mission: Impossible (1996)	51	5.41%	344	344	36.48%
Shawshank Redemption, The (1994)	50	5.30%	283	283	30.01%
Truth About Cats & Dogs, The (1996)	47	4.98%	272	272	28.84%
Godfather, The (1972)	47	4.98%	413	413	43.80%
Empire Strikes Back, The (1980)	46	4.88%	367	367	38.92%
Jaws (1975)	46	4.88%	280	280	29.69%
Mars Attacks! (1996)	46	4.88%	217	217	23.01%
Monty Python and the Holy Grail (1974)	45	4.77%	316	316	33.51%
Alien (1979)	45	4.77%	291	291	30.86%
Raiders of the Lost Ark (1981)	44	4.67%	420	420	44.54%

Figure 6.7: Popularity Bias

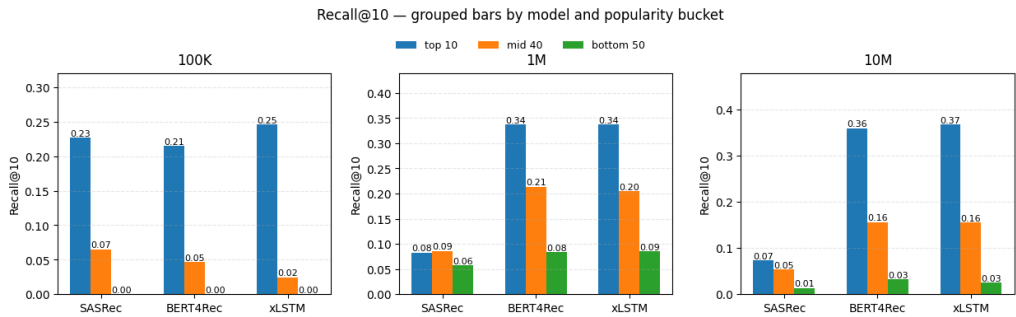


Figure 6.8: Popularity Chart

Table 6.2: Popularity Bucketed Metrics Across Models and Datasets

Model	Dataset	Seq Len	Bucket	Recall@10	MRR@10	NDCG@10
BERT4Rec	100K	128	top_10	0.2150	0.0714	0.1048
BERT4Rec	100K	128	mid_40	0.0471	0.0176	0.0243
BERT4Rec	100K	128	bottom_50	0.0000	0.0000	0.0000
BERT4Rec	1M	128	top_10	0.3373	0.1568	0.1991
BERT4Rec	1M	128	mid_40	0.2139	0.0797	0.1108
BERT4Rec	1M	128	bottom_50	0.0839	0.0376	0.0482
BERT4Rec	10M	128	top_10	0.3589	0.1728	0.2165
BERT4Rec	10M	128	mid_40	0.1562	0.0594	0.0818
BERT4Rec	10M	128	bottom_50	0.0321	0.0128	0.0172
SAS4Rec	100K	128	top_10	0.2272	0.0689	0.1055
SAS4Rec	100K	128	mid_40	0.0651	0.0178	0.0287
SAS4Rec	100K	128	bottom_50	0.0000	0.0000	0.0000
SAS4Rec	1M	128	top_10	0.0828	0.0354	0.0465
SAS4Rec	1M	128	mid_40	0.0863	0.0303	0.0433
SAS4Rec	1M	128	bottom_50	0.0579	0.0298	0.0362
SAS4Rec	10M	128	top_10	0.0730	0.0293	0.0394
SAS4Rec	10M	128	mid_40	0.0541	0.0211	0.0288
SAS4Rec	10M	128	bottom_50	0.0138	0.0066	0.0083
xLSTM	100K	128	top_10	0.2460	0.0806	0.1185
xLSTM	100K	128	mid_40	0.0246	0.0072	0.0112
xLSTM	100K	128	bottom_50	0.0000	0.0000	0.0000
xLSTM	1M	128	top_10	0.3369	0.1536	0.1964
xLSTM	1M	128	mid_40	0.2049	0.0776	0.1071
xLSTM	1M	128	bottom_50	0.0855	0.0377	0.0488
xLSTM	10M	128	top_10	0.3670	0.1818	0.2253
xLSTM	10M	128	mid_40	0.1551	0.0600	0.0820
xLSTM	10M	128	bottom_50	0.0254	0.0107	0.0141

trieval, making recommendations more consistent for similar users. [49, 23]

Group fairness: MovieLens contains demographic attributes, even though our model trains only on movie ID sequences. These underlying user group differences still sometimes influence interaction patterns, making popularity heavy models favor groups aligned with famous genres. Transformer embeddings sometimes might reinforce this by over emphasizing popular items. xLSTM’s more isotropic geometry reduces such dominance. This leads to more balanced item exposure across different user groups.[30, 54]

Popularity Calibration: Since MovieLens is highly head heavy, models must avoid over recommending already popular titles. Our results show

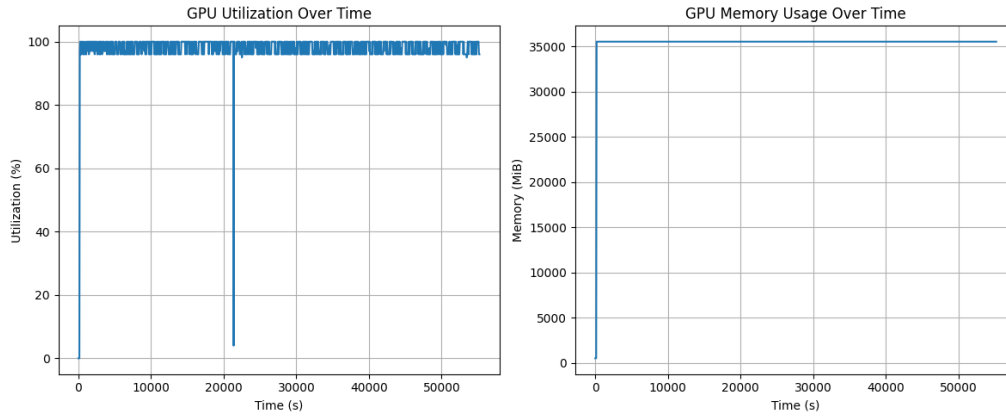


Figure 6.9: GPU Training Performance Graph

that xLSTM’s reduced hubness and higher intrinsic dimensionality produce a more calibrated popularity spread, aligning better with a user’s original preference mix than Transformer baselines. [48, 51]

6.6 Runtime Performance, Resource Usage and Logging

We have analyzed the measurements of GPU memory usage, training time and overall resource requirements for all five datasets. This analysis directly supports RQ3, helping compare the practicality of xLSTM vs. Transformer models.

We monitored GPU memory utilization, training time per epoch and inference latency for our five datasets. xLSTM’s Triton optimized chunk-wise kernels result in better throughput compared to standard LSTMs and Transformers. The total training time and peak memory usage are logged and benchmarked for each dataset scale (100K, 1M, 10M, Musik4all and Amazon Software).

To ensure the transparency and facilitate model debugging, to monitor the training metrics, we have used **TensorBoard**. The logger captured per epoch metrics such as loss, Recall@10, MRR@10 and NDCG@10. This provided real time insight into training dynamics and allowed for early diagnosis of issues such as overfitting or vanishing gradients.

For Model Checkpointing and Best Model Selection, a **model checkpointing strategy** was employed based on validation performance. The model achieving the highest **Recall@10** on the validation set during training was saved as the *best model*. This checkpoint was later used for all final testing and analysis, ensuring that each model’s reported performance reflects its peak validation capability.

6.7 Transfer Learning in Latent Factor Models

This section relate to RQ4 (embedding saturation), since transferability reflects how meaningful and general the learned embeddings are.

Using MovieLens 1M and 10M datasets, transfer learning involves training models (e.g., latent factor models or neural recommenders) on one dataset (like ML 1M), then adapting or fine tuning those models on the other dataset (like 10M). The key idea here is to transfer the learned embeddings and interaction patterns acquired from the smaller (or larger) dataset to boost performance on the other. This checks the common user item interaction structures that exist across both datasets. [41]

In earlier steps, we have learned latent features by performing various studies from saved best models, to understand the embedding dimensions and to find the similarities. Here in Transfer Learning, we will reuse such weights and analyses the performance on other similar domains or datasets. Item ID is the common joining conditions and we will maintain it uniform throughout across all other MovieLENS datasets. For example, available movies in 1M datasets will also have similar Item ID in other datasets as well, by which we will be comparing similar dimensional weights.

Such transfer is commonly done by reusing pretrained latent vectors or weights and fine tuning on the new dataset while addressing domain differences (e.g., different item catalogs). Our results demonstrate that models pretrained on MovieLens 1M can provide beneficial initialization for learning on the larger MovieLens 10M dataset and vice versa, fostering better accuracy and convergence speed. [41]

6.8 Industrial Applications

Here we interpret the results of RQ1 to RQ3 in the context of large scale deployments. The scaling differences observed across the 100K, 1M and 10M datasets demonstrate how xLSTM and Transformer models respond to increasing data volume, offering practical guidance on which architectures remain efficient, stable and accurate at industrial scale.

Recommender systems normally operate at massive scales and widely used in various sectors where millions of interactions accumulated. In ecommerce, streaming platforms, long user histories resemble ML1M and ML10M, making xLSTMs strong performance on long sequences operationally advantageous. In Finance and energy applications, where decisions depend on stable, and low latency models benefit from architectures like xLSTM that maintain accuracy while avoiding the quadratic overhead of attention. In domains such as healthcare, life science and service automation, fairness, robustness and consistency are critical, our RQ3 findings show that embedding compression in xLSTM remains more stable across data scales.

6.9 Limitations and Considerations

The conclusions which we drawn from RQ1 to RQ4 above be interpreted with few constraints of our experimental design, datasets and modeling scope. While xLSTM delivers strong performance across recommendations, few limitations might affect how these findings can be generalized.

Dataset Scope: Our results are based on MovieLens (100K, 10M) and MUSIK4All, which are static and domain specific. These datasets sometimes might miss dynamics catalog drift, session intent changes and scale so xLSTM's strong scaling trends may not fully generalize.

Sequence length Dependence: xLSTM performs best on long histories (ML1M/10M), but its gains weaken on short history datasets (ML100K, Amazon Software datasets). This limits the applicability to platforms with sparse or session only user interactions.

Embedding Utilization: Geometry findings from RQ4 (isotropy, higher intrinsic dimension) come from datasets with fixed vocabularies. Systems with millions of evolving items may produce different embeddings.

Production Gap: MovieLens and MUSIK4All do not reflect production feedback loops, popularity amplification, or adversarial interactions. Thus, our popularity bias and fairness interpretations may differ in live recommenders.

Despite these limitations, our results in chapter 5 provide clear metrics (RQ1 to RQ4) that xLSTM offers meaningful advantages in long sequence modeling, embedding utilization and representation diversity. These findings establish a strong foundation for future work on larger, more dynamic recommender datasets and indicate the potential for xLSTM based architectures in recommender systems.

Chapter 7

Conclusion

In this thesis, we have explored the design architecture, implementation, and evaluation of various sequential recommender systems using the extended LSTM (xLSTM) architecture. We have also benchmarked xLSTM against the strong baselines (such as SAS4Rec, BERT4Rec) across multiple public datasets (MovieLens 100K/1M/10M, MUSIK4All, Amazon Software). We have used evaluation metrics such as top k ranking metrics (Recall@10, MRR@10, NDCG@10), along with the inference speed runtime analysis and other qualitative recommendation outputs.

Our research findings shows that, xLSTM scales well with the longer interaction histories: on MovieLens 10M it well attains Recall@10 of around 0.318, comparable to the strong Transformer baselines results, and remains competitive as data size and sequence depth increase (e.g., 10M, MUSIK4All). At the same time, accuracy exhibits greater variance when sequence length grows further ($32 \rightarrow 64 \rightarrow 128$), indicating sensitivity to the chosen context window. While BERT4Rec remains consistently strong performance across the multiple settings, we found xLSTM offers the favorable memory scaling and stable training dynamics at the larger scale datasets, the trade-off here is higher computational cost, which is most noticeable on smaller datasets like MovieLens-100K where lighter Transformer variants can match or exceed both in accuracy along with lower runtime as discussed.

We have also deeply analyzed model parameters, model embedding dimensions, inference speed and also addressed practical challenges such as cold start problem, sparsity, and popularity bias through the sampling strategies. Embedding dimensions, popularity analyses, and qualitative reviews provided us the additional insight beyond the headline metrics and provided us in depth clarification where xLSTM benefits most in various application settings.

Overall, we verified that the xLSTM based approach forms the solid foundation for scalable sequential recommendation, which is strong at medium to large scale sequential datasets, memory efficient and training stable, while also acknowledging the reduced effectiveness on the smaller data regime and efficient enough for practical sequential recommender deployment.

Bibliography

- [1] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. *xLSTM: Extended Long Short-Term Memory*. 2024. arXiv: 2405.04517 [cs.LG].
- [2] D. Roy and M. Dutta. “A systematic review and research perspective on recommender systems”. In: *Journal of Big Data* 9.59 (2022). DOI: 10.1186/s40537-022-00592-5.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2017. arXiv: 1706.03762 [cs.CL].
- [4] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. “BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer”. In: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)* (2019), pp. 1441–1450. DOI: 10.1145/3357384.3357895.
- [5] Wang-Cheng Kang and Julian McAuley. “Self-Attentive Sequential Recommendation”. In: *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)* (2018), pp. 197–206. DOI: 10.1109/ICDM.2018.00035.
- [6] Markus Schedl, Peter Knees, Gerhard Widmer, et al. “MUSIK4all: A Large-Scale Music Dataset with User Interaction Histories”. In: *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*. 2022, pp. 887–894.
- [7] Deepak Kumar, Oleg Lesota, George Zerveas, Daniel Cohen, Carsten Eickhoff, Markus Schedl, and Navid Rekabsaz. “Parameter-efficient Modularised Bias Mitigation via Adapter Fusion”. In: *arXiv preprint arXiv:2302.06321* (2023).
- [8] Lukas Hauzenberger, Shahed Masoudian, Deepak Kumar, Markus Schedl, and Navid Rekabsaz. “Modular and On-demand Bias Mitigation with Attribute-Removal Subnetworks”. In: *arXiv preprint arXiv:2205.15171* (2022).
- [9] F. Maxwell Harper and Joseph A. Konstan. “The MovieLens Datasets: History and Context”. In: *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5.4 (2015), pp. 1–19. DOI: 10.1145/2827872.
- [10] Pablo Castells and Dietmar Jannach. *Recommender Systems: A Primer*. 2023. arXiv: 2302.02579 [cs.LG].
- [11] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

- [12] Dongfang Deng, Zixuan Lu, Qiang Liu, and Sepp Hochreiter. *xLSTM-Mixer: Multivariate Time Series Forecasting by Mixing via Scalar Memories*. 2024. arXiv: 2410.16928 [cs.LG].
- [13] Yifan Wu, Zixuan Lu, Dongfang Deng, and Sepp Hochreiter. *xLSTM Time: Long-term Time Series Forecasting With xLSTM*. 2024. arXiv: 2407.10240 [cs.LG].
- [14] Arpita Vats, Vinija Jain, Rahul Raja, and Aman Chadha. *Exploring the Impact of Large Language Models on Recommender Systems: An Extensive Review*. 2024. arXiv: 2402.18590 [cs.IR].
- [15] Jianmo Ni, Jiacheng Li, and Julian McAuley. “Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019, pp. 188–197. DOI: 10.18653/v1/D19-1018.
- [16] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. *Recommender Systems with Generative Retrieval*. 2024.
- [17] A. Noorian, A. Harounabadi, and M. Hazratifard. “A sequential neural recommendation system exploiting BERT and LSTM on social media posts”. In: *Complex Intelligent Systems* 10 (2024), pp. 721–744. DOI: 10.1007/s40747-023-01191-4.
- [18] H. Ahmadian Yazdi, S.J. Seyyed Mahdavi, and H. Ahmadian Yazdi. “Dynamic educational recommender system based on Improved LSTM neural network”. In: *Scientific Reports* 14 (2024), p. 4381.
- [19] X. Yang and J. A. Esquivel. “Time-Aware LSTM Neural Networks for Dynamic Personalized Recommendation on Business Intelligence”. In: *Tsinghua Science and Technology* 29.1 (2024), pp. 185–196. DOI: 10.26599/TST.2023.9010025.
- [20] Chhotelal Kumar and Mukesh Kumar. “Session-based recommendations with sequential context using attention-driven LSTM”. In: *Computers and Electrical Engineering* 115 (2024), p. 109138. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2024.109138.
- [21] Shivangi Gheewala, Shuxiang Xu, Soonja Yeom, and Sumbal Maqsood. “Exploiting deep transformer models in textual review based recommender systems”. In: *Expert Systems with Applications* (2023). DOI: 10.1016/j.eswa.2023.121120.
- [22] RecBole. *RecBole: A Unified Framework for Recommendation Algorithms*. <https://recbole.io>. Accessed: 2025-07-14. 2020.
- [23] Edward Raff. “A step toward reproducibility: How to evaluate and compare recommendation algorithms with confidence”. In: *Communications of the ACM* 64.4 (2021), pp. 72–80.
- [24] Matthew Hutson. “Artificial intelligence faces reproducibility crisis”. In: *Science* 359.6377 (2018), pp. 725–726.
- [25] Xing Zhang, Kuan Ren, Tao Zhang, and Mu Li. “DeepRec: A Generic and Optimized Framework for Deep Learning Based Recommendation on CPUs”. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2021, pp. 3209–3219.

- [26] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.
- [27] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. “Session-based recommendations with recurrent neural networks”. In: *International Conference on Learning Representations (ICLR)*. 2016.
- [28] Steffen Rendle, Li Zhang, Walid Krichene, and John Anderson. “Neural collaborative filtering vs. matrix factorization revisited”. In: *arXiv preprint arXiv:2005.09683* (2020).
- [29] Wayne Xin Zhao, Min Zhang, Junjie Wang, and Shaoping Ma. “Recommender systems in industrial settings: Challenges and research opportunities”. In: *Proceedings of the 43rd International ACM SIGIR Conference*. 2020, pp. 1361–1364.
- [30] Andrés Ferraro et al. “Break the Loop: Gender Imbalance in Music Recommenders”. In: *Proceedings of the ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR)*. Canberra, Australia, 2021.
- [31] Günther Ganhör et al. “Mitigating Consumer Biases in Recommendations with Adversarial Training”. In: *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. Madrid, Spain, 2022.
- [32] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. “Application of Dimensionality Reduction in Recommender System—A Case Study”. In: *Proceedings of the ACM WebKDD Workshop on Web Mining for E-Commerce*. 2000.
- [33] Yifan Hu, Yehuda Koren, and Chris Volinsky. “Collaborative Filtering for Implicit Feedback Datasets”. In: *Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM)*. 2008, pp. 263–272.
- [34] Jeffrey L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211.
- [35] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. “Learning to Forget: Continual Prediction with LSTM”. In: *Neural Computation* 12.10 (2000), pp. 2451–2471.
- [36] Massimo Quadrana, Paolo Cremonesi, and Dietmar Jannach. “Sequence-Aware Recommender Systems”. In: *ACM Computing Surveys* 51.4 (2018), 66:1–66:36.
- [37] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0.
- [38] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. Vol. 28. 3. 2013, pp. 1310–1318.
- [39] Paul Covington, Jay Adams, and Emre Sargin. “Deep Neural Networks for YouTube Recommendations”. In: *Proceedings of the 10th ACM Conference on Recommender Systems*. 2016.

- [40] Jonathan L. Herlocker, Joseph A. Konstan, Al Borchers, and John Riedl. "An Algorithmic Framework for Performing Collaborative Filtering". In: *Proceedings of the 22nd Annual International ACM SIGIR Conference* (1999).
- [41] Yehuda Koren. "Matrix factorization techniques for recommender systems". In: *Computer* 42.8 (2009), pp. 30–37.
- [42] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks*. Vol. 5. 2. 1994, pp. 157–166.
- [43] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. "Item-Based Collaborative Filtering Recommendation Algorithms". In: *Proceedings of the 10th International World Wide Web Conference*. 2001.
- [44] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *arXiv preprint arXiv:1412.6980* (2015).
- [45] Kalervo Järvelin and Jaana Kekäläinen. "Cumulated gain-based evaluation of IR techniques". In: *ACM Transactions on Information Systems* (2002).
- [46] Steffen Rendle et al. "BPR: Bayesian Personalized Ranking from Implicit Feedback". In: *UAI*. 2009.
- [47] Cai-Nicolas Ziegler, Sean M. McNee, Joseph A. Konstan, and Georg Lausen. "Improving Recommendation Lists Through Topic Diversification". In: *Proceedings of the 14th International World Wide Web Conference (WWW)*. 2005, pp. 22–32.
- [48] Saúl Vargas and Pablo Castells. "Rank and Relevance in Novelty and Diversity Metrics for Recommender Systems". In: *ACM Transactions on Information Systems* 29.4 (2011), pp. 1–32.
- [49] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. "Evaluating Collaborative Filtering Recommender Systems". In: *ACM Transactions on Information Systems* 22.1 (2004), pp. 5–53.
- [50] Sean M. McNee, John Riedl, and Joseph A. Konstan. "Being Accurate Is Not Enough: How Accuracy Metrics Have Hurt Recommender Systems". In: *CHI Extended Abstracts on Human Factors in Computing Systems*. 2006, pp. 1097–1101.
- [51] Gediminas Adomavicius and YoungOk Kwon. "Improving Aggregate Recommendation Diversity Using Ranking-Based Techniques". In: *IEEE Transactions on Knowledge and Data Engineering* 24.5 (2012), pp. 896–911.
- [52] Kawin Ethayarajh. "How Contextual Are Contextualized Word Representations?" In: *EMNLP* (2019).
- [53] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. "Hubs in space: Popular nearest neighbors in high-dimensional data". In: *Journal of Machine Learning Research* (2010).
- [54] Abdollahpouri et al. "Controlling popularity bias in recommender systems". In: *RecSys* (2017).

- [55] Laurens van der Maaten and Geoffrey Hinton. “Visualizing Data using t-SNE”. In: *JMLR* (2008).
- [56] Gao et al. “Representation Degeneration Problem in Neural Machine Translation”. In: *ICLR* (2019).
- [57] Wang et al. “On the Degeneration of Sentence Embeddings”. In: *EMNLP* (2020).
- [58] Li et al. “Measuring and Improving the Isotropy of Word Embeddings”. In: *arXiv* (2018).
- [59] Recanatesi et al. “Dimensionality Compression and Expansion in Deep Neural Networks”. In: *NeurIPS* (2019).
- [60] Dinu et al. “Improving Zero-Shot Learning by Mitigating the Hubness Problem”. In: *ICLR* (2015).
- [61] Yi Tay, Mostafa Dehghani, et al. “Efficient Transformers: A Survey”. In: *ACM Computing Surveys* (2020).
- [62] Iz Beltagy, Matthew Peters, and Arman Cohan. “Longformer: The Long-Document Transformer”. In: *arXiv preprint arXiv:2004.05150* (2020).
- [63] Sanjeev Arora et al. “A Latent Variable Model Approach to PMI-based Word Embeddings”. In: *Transactions of the ACL* (2016).
- [64] Yujia Li et al. “Dissecting the Geometry of Word Embeddings”. In: *arXiv* (2020).
- [65] Ansuini et al. “Intrinsic Dimensionality of Data Representations in Deep Neural Networks”. In: *NeurIPS* (2019).